Arquivos do Projeto DEV Platform

Conteúdo da pasta: ./src/stake_file

Gerado em: 19/06/2025 às 06:08

Índice

1. Arquivo: .env
2. Arquivo: .env.development
3. Arquivo: .env.production
4. Arquivo: .env.test
5. Arquivo: .gitignore
6. Arquivo: alembic.ini
7. Arquivo: composition_root.py
8. Arquivo: config.production.json
9. Arquivo: config.py
10. Arquivo: dtos.py
11. Arquivo: entities.py
12. Arquivo: env.py
13. Arquivo: exceptions.py
14. Arquivo: interfaces.py
15. Arquivo: logger.py
16. Arquivo: main.py
17. Arquivo: mappers.py
18. Arquivo: mkdocs.yml
19. Arquivo: models.py
20. Arquivo: mypy.ini
21. Arquivo: poetry.toml
22. Arquivo: ports.py
23. Arquivo: pyproject.toml
24. Arquivo: README.md
25. Arquivo: repositories.py
26. Arquivo: services.py
27. Arquivo: session.py
28. Arquivo: settings.json
29. Arquivo: structured_logger.py
30. Arquivo: tree.txt
31. Arquivo: unit_of_work.py
32. Arquivo: use_cases.py
33. Arquivo: user_commands.py
34. Arquivo: user_exceptions.py
35. Arquivo: validation_rules.py
36. Arquivo: value_objects.py

1. Arquivo: .env

```
# ./.env
DB_USER_REMOTE="root"
DB_PASSWORD_REMOTE="Malato%2301"
DB_HOST_REMOTE="127.0.0.1:3306"
DB_NAME_REMOTE="env_management_db"

logging_level=DEBUG
```

2. Arquivo: .env.development

```
# ./.env.development
# Nota 1: Este arquivo contém as variáveis de ambiente para o ambiente de desenvolvimento.
# Certifique-se de que este arquivo não seja incluído no controle de versão, pois contém informações sen
síveis.
# Nota 2: Essas variáveis de ambiente estão em fase de transferência para melhor gestão de segredos em
 produção e para
# otimizar a validação de dados, visando maior segurança e eficiência.

# Configurações de ambiente para desenvolvimento
ENVIRONMENT=development
DATABASE_URL=mysql+aiomysql://root:Malato%2301@127.0.0.1:3306/user_management
DB_POOL_SIZE=5
DB_MAX_OVERFLOW=10
DB_ECHO=True
LOG_LEVEL=DEBUG
DEBUG=True

ALLOWED_DOMAINS="dominio1.com, dominio2.com"
VALIDATION_FORBIDDEN_WORDS="palavra1, palavra2"
ENTERPRISE_FORBIDDEN_WORDS="palavra3, palavra4"
ENTERPRISE_ALLOWED_DOMAINS="dominio3.com, dominio4.com"
```

3. Arquivo: .env.production

```
# ./.env.production
# Nota 1: Este arquivo contém as variáveis de ambiente para o ambiente de produção.
# Certifique-se de que este arquivo não seja incluído no controle de versão, pois contém informações sen
síveis.
# Nota 2 Essas variáveis de ambiente estão em fase de trasferência para melhor gestão de segredos em p
rodução e para
# otimizar a validação de dados, visando maior segurança e eficiência.

# Configurações de ambiente para produção
ENVIRONMENT=production
DATABASE_URL=mysql+aiomysql://root:Malato%2301@127.0.0.1:3306/user_management
DB_POOL_SIZE=20
DB_MAX_OVERFLOW=30
DB_ECHO=False
LOG_LEVEL=INFO
DEBUG=False

# Configurações de validação
VALIDATION_ENABLE_PROFANITY_FILTER=True
VALIDATION_FORBIDDEN_WORDS="palavrao1, palavrao2, termo_proibido" # Lista de palavras proibidas,
separadas por vírgula
VALIDATION_ALLOWED_DOMAINS="dominio1.com, dominio2.com" # Lista de domínios permitidos, sepa
radas por vírgula
VALIDATION_BUSINESS_HOURS_ONLY=False


# Configurações de domínios permitidos para empresas
ENTERPRISE_ALLOWED_DOMAINS="empresa.com, company.com"

ALLOWED_DOMAINS=
ENTERPRISE_FORBIDDEN_WORDS=
ENTERPRISE_ALLOWED_DOMAINS=
```

4. Arquivo: .env.test

```
# ./.env.test
# Nota 1: Este arquivo contém as variáveis de ambiente para o ambiente de teste.
# Certifique-se de que este arquivo não seja incluído no controle de versão, pois contém informações sen
síveis.
# Nota 2: Essas variáveis de ambiente estão em fase de transferência para melhor gestão de segredos em
 produção e para
# otimizar a validação de dados, visando maior segurança e eficiência.

# Configurações de ambiente para testes
ENVIRONMENT=test
DATABASE_URL=mysql+aiomysql://root:Malato%2301@127.0.0.1:3306/user_management
DB_POOL_SIZE=1
DB_MAX_OVERFLOW=0
DB_ECHO=False
LOG_LEVEL=WARNING
DEBUG=False

ALLOWED_DOMAINS="dominio1.com, dominio2.com"
VALIDATION_FORBIDDEN_WORDS="palavra1, palavra2"
ENTERPRISE_FORBIDDEN_WORDS="palavra3, palavra4"
ENTERPRISE_ALLOWED_DOMAINS="dominio3.com, dominio4.com"
```

5. Arquivo: .gitignore

```
# ./.gitignore


# Ambientes virtuais
.venv/
venv/
ENV/
env/

# Arquivos de cache Python
__pycache__/
*.py[cod]
*$py.class
.pytest_cache/
.coverage
htmlcov/
.tox/
.nox/

# Distribuição / empacotamento
dist/
build/
*.egg-info/
*.egg

# Documentação gerada pelo MkDocs
site/

# Poetry
poetry.lock
# NOTA:
# 1 - Descomente a linha acima se NÃO quiser versionar o poetry.lock
# 2 - Commit do arquivo poetry.lock permite criar um ambiente determinístico

# Arquivos de log
*.log
logs/

# Arquivos temporários
*.tmp
*.bak
*.swp
*~

# Arquivos do sistema operacional
.DS_Store
Thumbs.db

# IDEs e editores
.idea/
.vscode/
*.sublime-project
*.sublime-workspace
.project
.pydevproject
.spyderproject
.spyproject
.ropeproject

# Jupyter Notebook
.ipynb_checkpoints

# Arquivos de ambiente
```

```
.env
.env.local
.env.development.local
.env.test.local
.env.production.local
.env.development
.env.test
.env.production
.env*

# Diretórios de mídia/uploads (se aplicável)
media/
uploads/

# SQLite DB (se aplicável)
*.sqlite3
*.db

# Arquivos de configuração local
local_settings.py
```

6. Arquivo: alembic.ini

```
# ./alembic.ini
# A generic, single database configuration.

[alembic]
# path to migration scripts
# Use forward slashes (/) also on windows to provide an os agnostic path
script_location = migrations

# template used to generate migration file names; The default value is %%(rev)s_%%(slug)s
# Uncomment the line below if you want the files to be prepended with date and time
# see https://alembic.sqlalchemy.org/en/latest/tutorial.html#editing-the-ini-file
# for all available tokens
# file_template = %%(year)d_%%(month).2d_%%(day).2d_%%(hour).2d%%(minute).2d-%%(rev)s_%%(slug)
s

# sys.path path, will be prepended to sys.path if present.
# defaults to the current working directory.
prepend_sys_path = .

# timezone to use when rendering the date within the migration file
# as well as the filename.
# If specified, requires the python>=3.9 or backports.zoneinfo library and tzdata library.
# Any required deps can installed by adding `alembic[tz]` to the pip requirements
# string value is passed to ZoneInfo()
# leave blank for localtime
# timezone =

# max length of characters to apply to the "slug" field
# truncate_slug_length = 40

# set to 'true' to run the environment during
# the 'revision' command, regardless of autogenerate
# revision_environment = false

# set to 'true' to allow .pyc and .pyo files without
# a source .py file to be detected as revisions in the
# versions/ directory
# sourceless = false

# version location specification; This defaults
# to migrations/versions.  When using multiple version
# directories, initial revisions must be specified with --version-path.
# The path separator used here should be the separator specified by "version_path_separator" below.
# version_locations = %(here)s/bar:%(here)s/bat:migrations/versions

# version path separator; As mentioned above, this is the character used to split
# version_locations. The default within new alembic.ini files is "os", which uses os.pathsep.
# If this key is omitted entirely, it falls back to the legacy behavior of splitting on spaces and/or commas.
# Valid values for version_path_separator are:
#
# version_path_separator = :
# version_path_separator = ;
# version_path_separator = space
# version_path_separator = newline
#
# Use os.pathsep. Default configuration used for new projects.
version_path_separator = os

# set to 'true' to search source files recursively
# in each "version_locations" directory
# new in Alembic version 1.10
# recursive_version_locations = false
```

```
# the output encoding used when revision files
# are written from script.py.mako
# output_encoding = utf-8

sqlalchemy.url = "mysql+aiomysql://root:Malato#01@127.0.0.1:3306/user_management" # driver://user:p
ass@localhost/dbname

[post_write_hooks]
# post_write_hooks defines scripts or Python functions that are run
# on newly generated revision scripts.  See the documentation for further
# detail and examples

# format using "black" - use the console_scripts runner, against the "black" entrypoint
# hooks = black
# black.type = console_scripts
# black.entrypoint = black
# black.options = -l 79 REVISION_SCRIPT_FILENAME

# lint with attempts to fix using "ruff" - use the exec runner, execute a binary
# hooks = ruff
# ruff.type = exec
# ruff.executable = %(here)s/.venv/bin/ruff
# ruff.options = check --fix REVISION_SCRIPT_FILENAME

# Logging configuration
[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARNING
handlers = console
qualname =

[logger_sqlalchemy]
level = WARNING
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
datefmt = %H:%M:%S
```

7. Arquivo: composition_root.py

```
# ./src/dev_platform/infrastructure/composition_root.py
# -*- coding: utf-8 -*-
"""
Este módulo define a raiz de composição para injeção de dependências,
centralizando a criação e configuração de todas as dependências da aplicação.
Agora utiliza um ValidationRuleProvider para desacoplar a lógica de regras de validação,
respeitando OCP e separando logging de infraestrutura.
"""

from typing import Any, Dict, List, Optional
from dev_platform.infrastructure.config import ConfigurationFacade
from dev_platform.application.user.use_cases import (
    CreateUserUseCase,
    ListUsersUseCase,
    UpdateUserUseCase,
    GetUserUseCase,
    DeleteUserUseCase,
)
from dev_platform.domain.user.interfaces import IUserRepository
from dev_platform.infrastructure.database.unit_of_work import SQLUnitOfWork
from dev_platform.infrastructure.logging.structured_logger import StructuredLogger
from dev_platform.domain.user.services import (
    UserDomainService,
    UserAnalyticsService,
)
from dev_platform.domain.validation_rules import (
    EmailFormatAdvancedValidationRule,
    NameContentValidationRule,
    EmailDomainValidationRule,
    BusinessHoursValidationRule,
    NameProfanityValidationRule,
    ForbiddenWordsValidationRule
)
from dev_platform.application.ports.logger import ILogger
from dev_platform.domain.validation_rules import ValidationRule


class ValidationRuleProvider:
    """
    Provider para regras de validação de usuários.
    Permite extensão sem modificar a CompositionRoot (OCP).
    Responsável por logging de configuração relacionado às regras.
    """
    def __init__(self, config: Dict[str, Any], logger: ILogger):
        self._config = ConfigurationFacade()
        self._logger = logger

    def get_rules(self, user_type: str = "default") -> List[ValidationRule]:
        """
        Retorna a lista de regras de validação conforme o tipo de usuário.
        """
        if user_type == "enterprise":
            return self._enterprise_rules()
        return self._default_rules()

    def _default_rules(self) -> List[ValidationRule]:
        allowed_domains = self._parse_csv("allowed_domains")
        forbidden_words = self._parse_csv("validation_forbidden_words")
        validation_config = self._config.get("validation", {})

        if validation_config.get("enable_profanity_filter", False) and not forbidden_words:
            self._logger.warning("Profanity filter enabled, but forbidden words list is empty in configuration.")
```

```
        return [
            EmailFormatAdvancedValidationRule(),
            NameContentValidationRule(),
            EmailDomainValidationRule(allowed_domains),
            BusinessHoursValidationRule(validation_config.get("business_hours_only", False)),
            NameProfanityValidationRule(forbidden_words=forbidden_words),
            ForbiddenWordsValidationRule(forbidden_words=forbidden_words)
        ]

    def _enterprise_rules(self) -> List[ValidationRule]:
        validation_forbidden_words = self._parse_csv("validation_forbidden_words")
        enterprise_forbidden_words = self._parse_csv("enterprise_forbidden_words")
        enterprise_allowed_domains = self._parse_csv("enterprise_allowed_domains")

        if not enterprise_forbidden_words:
            self._logger.warning("Enterprise forbidden words list is empty in configuration.")

        return [
            ForbiddenWordsValidationRule(validation_forbidden_words),
            NameProfanityValidationRule(enterprise_forbidden_words),
            EmailDomainValidationRule(enterprise_allowed_domains),
            BusinessHoursValidationRule(True),
        ]

    def _parse_csv(self, key: str) -> List[str]:
        value = self._config.get(key)
        if not value:
            return []
        return [w.strip() for w in value.split(",") if w.strip()]


class CompositionRoot:
    """
    Composition root for dependency injection.
    Centraliza a criação e configuração de todas as dependências da aplicação.
    Utiliza ValidationRuleProvider para regras de validação (OCP).
    """

    def __init__(
        self,
        environment: str,
        logger: Optional[ILogger] = None,
        validation_rule_provider: Optional[ValidationRuleProvider] = None
    ):
        self._environment = environment
        self._config = ConfigurationFacade()
        self._logger = logger or StructuredLogger()
        self._validation_rule_provider = validation_rule_provider or ValidationRuleProvider(self._config, self._l
ogger)

    def create_user_use_case(self, uow: SQLUnitOfWork, user_repository: IUserRepository) -> CreateUserUs
eCase:
        return CreateUserUseCase(
            uow=uow,
            domain_service=self.user_domain_service(user_repository),
            logger=self._logger,
        )

    def list_users_use_case(self, uow: SQLUnitOfWork) -> ListUsersUseCase:
        return ListUsersUseCase(uow=uow, logger=self._logger)

    def update_user_use_case(self, uow: SQLUnitOfWork, user_repository: IUserRepository) -> UpdateUser
UseCase:
        return UpdateUserUseCase(
            uow=uow,
            domain_service=self.user_domain_service(user_repository),
```

```
            logger=self._logger,
        )

    def get_user_use_case(self, uow: SQLUnitOfWork, user_repository: IUserRepository) -> GetUserUseCase:
        return GetUserUseCase(
            uow=uow,
            domain_service=self.user_domain_service(user_repository),
            logger=self._logger,
        )

    def delete_user_use_case(self, uow: SQLUnitOfWork, user_repository: IUserRepository) -> DeleteUserUs
eCase:
        return DeleteUserUseCase(
            uow=uow,
            domain_service=self.user_domain_service(user_repository),
            logger=self._logger,
        )

    def user_domain_service(self, user_repository: IUserRepository, user_type: str = "default") -> UserDomai
nService:
        """
        Cria UserDomainService com regras de validação baseadas em configuração e tipo de usuário.
        """
        rules = self._validation_rule_provider.get_rules(user_type)
        return UserDomainService(user_repository, rules)

    def user_analytics_service(self, user_repository: IUserRepository) -> UserAnalyticsService:
        """
        Cria o serviço de analytics de usuários.
        """
        return UserAnalyticsService(user_repository)

    def create_enterprise_user_domain_service(
        self, user_repository: IUserRepository
    ) -> UserDomainService:
        return self.user_domain_service(user_repository, user_type="enterprise")
```

8. Arquivo: config.production.json

```
{
    "name": "dev_platform",
    "version": "1.0.0",
    "description": "Production configuration for the development platform",
    "environment": "production",
    "apiBaseUrl": "https://api.devplatform.com",
    "loggingLevel": "error",
    "features": {
        "enableFeatureX": false,
        "enableFeatureY": true
    },
    "database": {
        "databasename": "user_management",
        "host": "127.0.0.1",
        "port": 3306,
        "username": "root",
        "password": "Malato%2301"
    }
}
```

9. Arquivo: config.py

```
# ./src/dev_platform/infrastructure/config.py
# -*- coding: utf-8 -*-
"""
Este módulo define a infraestrutura de configuração do DEV Platform,
responsável por carregar e acessar as configurações do sistema.
"""

import os
import json
from typing import Dict, Any, Optional, Callable, Type
from dotenv import load_dotenv
from dev_platform.domain.exceptions import ConfigurationException
from dev_platform.application.ports.logger import ILogger
from dev_platform.infrastructure.logging.structured_logger import StructuredLogger

class EnvLoader:
    """
    Responsável por carregar variáveis de ambiente de arquivos .env.
    """
    def __init__(self, environment: str, logger: ILogger):
        self.environment = environment
        self.logger = logger

    def load(self) -> None:
        """
        Carrega variáveis de ambiente do arquivo .env.<environment>.
        """
        dotenv_path = f".env.{self.environment}"
        base_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", ".."))
        full_dotenv_path = os.path.join(base_dir, dotenv_path)
        if os.path.exists(full_dotenv_path):
            load_dotenv(dotenv_path=full_dotenv_path, override=True)
        else:
            if self.environment == "production":
                self.logger.info(
                    f"Arquivo .env.{self.environment} não encontrado em {full_dotenv_path}. Assumindo que as v
ariáveis de ambiente são configuradas externamente."
                )
            else:
                self.logger.info(
                    f"AVISO: Arquivo .env.{self.environment} não encontrado em {full_dotenv_path}. Algumas vari
áveis de ambiente podem não estar definidas."
                )

class JsonConfigLoader:
    """
    Responsável por carregar configurações de arquivos JSON.
    """
    def __init__(self, environment: str, logger: ILogger):
        self.environment = environment
        self.logger = logger

    def load(self) -> Dict[str, Any]:
        """
        Carrega configurações do arquivo config.<environment>.json.
        """
        config_file_path = f"config.{self.environment}.json"
        base_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "..", ".."))
        full_config_file_path = os.path.join(base_dir, config_file_path)
        config: Dict[str, Any] = {}
        if os.path.exists(full_config_file_path):
            try:
                with open(full_config_file_path, "r") as f:
```

```
                config = json.load(f)
            except Exception as e:
                self.logger.error(
                    "Erro ao carregar o arquivo de configuração.",
                    config_key="CONFIG_FILE_LOAD_ERROR"
                )
                raise ConfigurationException(
                    config_key="CONFIG_FILE_LOAD_ERROR",
                    reason=f"Erro ao carregar o arquivo de configuração {full_config_file_path}: {e}"
                )
        else:
            self.logger.info(
                f"Arquivo de {full_config_file_path} não encontrado. Usando apenas variáveis de ambiente e pa
drões.",
                config_key="CONFIG_FILE_NOT_FOUND"
            )
        return config

class ConfigValidator:
    """
    Responsável por validar configurações críticas.
    """
    def __init__(self, environment: str, logger: ILogger):
        self.environment = environment
        self.logger = logger

    def validate(self) -> None:
        """
        Valida se as configurações críticas estão presentes (ex: DATABASE_URL em produção).
        """
        if self.environment == "production":
            if not os.getenv("DATABASE_URL"):
                self.logger.error(
                    "DATABASE_URL não configurada para produção.",
                    config_key="DATABASE_URL"
                )
                raise ConfigurationException(
                    config_key="DATABASE_URL",
                    reason="DATABASE_URL must be set in production environment."
                )

class DatabaseDriverChecker:
    """
    Responsável por garantir o uso de drivers assíncronos na URL do banco de dados.
    """
    @staticmethod
    def ensure_async_driver(url: str) -> str:
        """
        Ajusta a URL do banco para usar driver assíncrono, se necessário.
        """
        if url.startswith("mysql://"):
            return url.replace("mysql://", "mysql+aiomysql://")
        elif url.startswith("postgresql://"):
            return url.replace("postgresql://", "postgresql+asyncpg://")
        elif url.startswith("sqlite:///"):
            return url.replace("sqlite:///", "sqlite+aiosqlite:///")
        return url

class ConfigAccessor:
    """
    Responsável por acessar valores de configuração.
    """
    def __init__(self, config: Dict[str, Any], logger: ILogger):
        self._config = config
        self._logger = logger
```

```
    def get(self, key: str, default: Any = None) -> Any:
        """
        Obtém um valor de configuração, preferindo variáveis de ambiente.
        """
        env_key = key.upper().replace(".", "_")
        env_value = os.getenv(env_key)
        if env_value is not None:
            return env_value
        value = self._config.get(key, default)
        if value is None:
            self._logger.error(
                f"Configuração obrigatória '{key}' não encontrada.",
                config_key=key
            )
            raise ConfigurationException(
                config_key=key,
                reason=f"Required configuration '{key}' is missing."
            )
        return value

    def get_typed(self, key: str, default: Any = None, cast_type: Type = str) -> Any:
        """
        Obtém um valor de configuração e converte para o tipo desejado.
        """
        value = self.get(key, default)
        if value is None:
            return default
        try:
            if cast_type is bool:
                return str(value).strip().lower() in ("1", "true", "yes", "on")
            return cast_type(value)
        except Exception:
            return default

    def get_all_config(self) -> Dict[str, Any]:
        """
        Retorna todas as configurações efetivas, mesclando arquivo JSON e variáveis de ambiente.
        """
        all_configs = self._config.copy()
        for env_key, env_value in os.environ.items():
            all_configs[env_key.lower().replace(".", "_")] = env_value
        return all_configs

class ConfigurationFacade:
    """
    Fachada para acesso às configurações do sistema, permitindo injeção de dependências.

    Parâmetros de __init__:
        logger: ILogger customizado (opcional)
        env_loader_factory: Callable para criar um EnvLoader customizado (opcional)
        json_loader_factory: Callable para criar um JsonConfigLoader customizado (opcional)
        validator_factory: Callable para criar um ConfigValidator customizado (opcional)
        accessor_factory: Callable para criar um ConfigAccessor customizado (opcional)
    """
    _instance: Optional["ConfigurationFacade"] = None

    def __new__(cls, *args, **kwargs) -> "ConfigurationFacade":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(
        self,
        logger: Optional[ILogger] = None,
        env_loader_factory: Optional[Callable[[str, ILogger], EnvLoader]] = None,
        json_loader_factory: Optional[Callable[[str, ILogger], JsonConfigLoader]] = None,
```

```
        validator_factory: Optional[Callable[[str, ILogger], ConfigValidator]] = None,
        accessor_factory: Optional[Callable[[Dict[str, Any], ILogger], ConfigAccessor]] = None,
        environment: Optional[str] = None,
    ) -> None:
        """
        Inicializa a fachada de configuração, permitindo injeção de dependências para testes ou customizaçã
o.
        """
        if hasattr(self, "_initialized") and self._initialized:
            return
        self._logger: ILogger = logger or StructuredLogger()
        self._environment: str = environment or os.getenv("ENVIRONMENT", "production")

        # Permite injeção de dependências para facilitar testes/mocks
        env_loader: EnvLoader = (env_loader_factory or EnvLoader)(self._environment, self._logger)
        env_loader.load()
        json_loader: JsonConfigLoader = (json_loader_factory or JsonConfigLoader)(self._environment, self._l
ogger)
        config_dict: Dict[str, Any] = json_loader.load()
        validator: ConfigValidator = (validator_factory or ConfigValidator)(self._environment, self._logger)
        validator.validate()
        self._accessor: ConfigAccessor = (accessor_factory or ConfigAccessor)(config_dict, self._logger)
        self._initialized: bool = True

    @classmethod
    def _reset_singleton(cls) -> None:
        """
        Reseta a instância singleton (apenas para uso em testes).
        """
        cls._instance = None

    def get(self, key: str, default: Any = None) -> Any:
        """
        Obtém um valor de configuração, preferindo variáveis de ambiente.
        """
        return self._accessor.get(key, default)

    def get_typed(self, key: str, default: Any = None, cast_type: Type = str) -> Any:
        """
        Obtém um valor de configuração e converte para o tipo desejado.
        """
        return self._accessor.get_typed(key, default, cast_type)

    def get_all_config(self) -> Dict[str, Any]:
        """
        Retorna todas as configurações efetivas, mesclando arquivo JSON e variáveis de ambiente.
        """
        return self._accessor.get_all_config()

    @property
    def database_url(self) -> str:
        """
        Retorna a URL do banco de dados com driver assíncrono garantido.
        """
        url = self.get("DATABASE_URL")
        if not url:
            raise ConfigurationException(
                config_key="DATABASE_URL",
                reason="DATABASE_URL is not configured for the current environment."
            )
        return DatabaseDriverChecker.ensure_async_driver(url)

    @property
    def sync_database_url(self) -> str:
        """
        Retorna a URL do banco de dados sem garantir driver assíncrono.
```

```
"""
url = self.get("DATABASE_URL")
if not url:
    raise ConfigurationException(
        config_key="DATABASE_URL",
        reason="DATABASE_URL is not configured for the current environment."
    )
return url
```

10. Arquivo: dtos.py

```
# ./src/dev_platform/application/user/dtos.py
# -*- coding: utf-8 -*-
"""
Este módulo define os Data Transfer Objects (DTOs) para a entidade User,
permitindo a transferência de dados entre camadas da aplicação.
"""

from pydantic import BaseModel, StrictStr, EmailStr, field_validator

class UserDTO(BaseModel):
    """
    Data Transfer Object for User entity.
    """
    id: StrictStr
    name: StrictStr
    email: StrictStr

    class Config:
        """
        Configurações do Pydantic para o DTO.
        """
        from_attributes = True

class UserCreateDTO(BaseModel):
    """
    Data Transfer Object for creating a new User.
    """
    name: StrictStr
    email: EmailStr

    @field_validator("name", mode="before")
    def validate_name(cls, v):
        """
        Valida o nome do usuário.
        """
        if not v or len(v) == 0:
            raise ValueError("Precisa ser um nome, o campo não pode ficar vazio")
        return v.strip()

    @field_validator("email", mode="before")
    def validate_email(cls, v):
        """
        Valida o e-mail do usuário.
        """
        if not v or len(v) == 0:
            raise ValueError("Precisa ser um e-mail")
        return v.lower().strip()

    class Config:
        """
        Configurações do Pydantic para o DTO de criação.
        """
        from_attributes = True

class UserUpdateDTO(BaseModel):
    """
    Data Transfer Object for updating an existing User.
    """
    name: StrictStr
    email: EmailStr

    @field_validator("name", mode="before")
    def validate_name(cls, v):
```

```
    """
    Valida o nome do usuário para atualização.
    """
    return v.strip()

@field_validator("email", mode="before")
def validate_email(cls, v):
    """
    Valida o e-mail do usuário para atualização.
    """
    return v.lower().strip()

class Config:
    """
    Configurações do Pydantic para o DTO de atualização.
    """
    from_attributes = True
```

11. Arquivo: entities.py

```
# ./src/dev_platform/domain/user/entities.py
# -*- coding: utf-8 -*-
"""
Este módulo define a entidade User, representando um usuário do sistema.
A entidade é imutável e utiliza Value Objects para validação de nome e e-mail.
"""

from dataclasses import dataclass, replace
from typing import Optional
from dev_platform.domain.user.value_objects import Email, UserName

@dataclass(frozen=True)
class User:
    """Entidade de domínio representando um usuário (imutável)."""
    id: Optional[int]
    name: UserName
    email: Email

    @classmethod
    def create(cls, name: str, email: str) -> "User":
        """Cria um novo usuário, validando nome e e-mail via Value Objects."""
        return cls(id=None, name=UserName(name), email=Email(email))

    def with_id(self, new_id: int) -> "User":
        """Retorna uma nova instância do usuário com o id atualizado."""
        return replace(self, id=new_id)

    def update_details(self, new_name: str, new_email: str) -> "User":
        """Retorna uma nova instância do usuário com nome e e-mail atualizados."""
        return replace(self, name=UserName(new_name), email=Email(new_email))
```

12. Arquivo: env.py

```
# ./migrations/env.py
from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool
from alembic import context

# Isso importa a instância singleton da sua configuração
# Assumindo que o caminho é src.dev_platform.infrastructure.config
# Ajuste o import path conforme a estrutura real do seu projeto.
import os
import sys

# Adiciona o diretório raiz do projeto ao sys.path para imports absolutos funcionarem
# Se alembic.ini e .env estiverem na raiz do projeto, e src/dev_platform for o módulo,
# precisamos garantir que o sys.path inclua o diretório que contém 'src'.
# O Alembic geralmente é executado da raiz do projeto, então isso é crucial.
# Descobre o caminho do alembic.ini e navega para a raiz do projeto.
current_dir = os.path.dirname(os.path.abspath(__file__))
project_root = os.path.abspath(
    os.path.join(current_dir, "..", "..")
)  # Ajuste conforme a profundidade de 'migrations'
sys.path.insert(0, project_root)

# Agora podemos importar a configuração do seu projeto
# Ajuste o caminho se a sua `config.py` não estiver diretamente em infrastructure
from src.dev_platform.infrastructure.config import CONFIG, ConfigurationException

# This is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

# Interpret the config file for Python logging.
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import Base
# target_metadata = Base.metadata
# Exemplo: Se você tem um models.py na camada de infraestrutura
from src.dev_platform.infrastructure.database.models import (
    Base,
)  # Ajuste este import para seu modelo base

target_metadata = Base.metadata

# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.


# --- Começo da Modificação para usar sua CONFIG ---
def get_database_url_from_project_config():
    try:
        # Acesse a URL do banco de dados diretamente da sua instância CONFIG
        # Isso garante que a URL seja carregada dos arquivos .env corretamente.
        return (
            CONFIG.sync_database_url
        )  # Use sync_database_url para Alembic, pois ele é síncrono.
    except ConfigurationException as e:
```

```
        print(
            f"ERRO: Não foi possível obter DATABASE_URL da configuração do projeto: {e}"
        )
        # Isso é um erro fatal para o Alembic, então re-lançar ou sair.
        sys.exit(1)


# Pega a URL do banco de dados da sua configuração customizada
database_url = get_database_url_from_project_config()

# Define a URL do banco de dados para o contexto do Alembic
config.set_main_option("sqlalchemy.url", database_url)

# --- Fim da Modificação ---


def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
    and not an Engine, though an Engine is additionally
    permissible.  By not creating an Engine, we don't even
    need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()


def run_migrations_online() -> None:
    """Run migrations in 'online' mode.

    In this scenario we need to create an Engine
    and associate a connection with the context.

    """
    # config.url já está definido pelo get_database_url_from_project_config() acima
    connectable = engine_from_config(
        CONFIG.get_section_arg(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata)

        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

13. Arquivo: exceptions.py

```
# ./src/dev_platform/domain/exceptions.py
# -*- coding: utf-8 -*-
"""
Este módulo define exceções para a camada de aplicação e infraestrutura,
permitindo o tratamento de erros de forma consistente e informativa.
"""

from datetime import datetime
from typing import Optional, Dict, Any

# Application layer exceptions
class ApplicationException(Exception):
    """Base exception for application layer errors."""

    def __init__(self, message: str, original_exception: Optional[Exception] = None):
        self.message = message
        self.original_exception = original_exception
        self.timestamp = datetime.now()
        super().__init__(self.message)

    def __str__(self):
        return f"[ApplicationException] {self.message}"

class UseCaseException(ApplicationException):
    """Raised when a use case execution fails."""

    def __init__(
        self,
        use_case_name: str,
        reason: str,
        original_exception: Optional[Exception] = None,
    ):
        self.use_case_name = use_case_name
        self.reason = reason
        super().__init__(
            message=f"Use case '{use_case_name}' failed: {reason}",
            original_exception=original_exception,
        )

    def __str__(self):
        return f"[UseCaseException] {self.message}"

# Infrastructure layer exceptions
class InfrastructureException(Exception):
    """Base exception for infrastructure layer errors."""

    def __init__(
        self,
        message: str,
        component: str,
        original_exception: Optional[Exception] = None,
    ):
        self.message = message
        self.component = component
        self.original_exception = original_exception
        self.timestamp = datetime.now()
        super().__init__(self.message)

    def to_dict(self) -> Dict[str, Any]:
        """Convert exception to dictionary for logging/serialization."""
        return {
            "message": self.message,
            "component": self.component,
```

```
            "timestamp": self.timestamp.isoformat(),
            "original_error": str(self.original_exception)
            if self.original_exception
            else None,
        }

    def __str__(self):
        return f"[InfrastructureException:{self.component}] {self.message}"

class DatabaseException(InfrastructureException):
    """Raised when database operations fail."""

    def __init__(
        self,
        operation: str,
        reason: str,
        original_exception: Optional[Exception] = None,
    ):
        self.operation = operation
        self.reason = reason
        super().__init__(
            message=f"Database operation '{operation}' failed: {reason}",
            component="database",
            original_exception=original_exception,
        )

    def to_dict(self) -> Dict[str, Any]:
        """Extended dictionary representation for database errors."""
        base_dict = super().to_dict()
        base_dict.update({"operation": self.operation, "reason": self.reason})
        return base_dict

    def __str__(self):
        return f"[DatabaseException:{self.operation}] {self.message}"

class ConfigurationException(InfrastructureException):
    """Raised when configuration is invalid or missing."""

    def __init__(self, config_key: str, reason: str):
        self.config_key = config_key
        self.reason = reason
        super().__init__(
            message=f"Configuration error for '{config_key}': {reason}",
            component="configuration",
        )

    def __str__(self):
        return f"[ConfigurationException:{self.config_key}] {self.message}"

class CacheException(InfrastructureException):
    """Raised when cache operations fail."""

    def __init__(
        self,
        operation: str,
        key: str,
        reason: str,
        original_exception: Optional[Exception] = None,
    ):
        self.operation = operation
        self.key = key
        self.reason = reason
        super().__init__(
            message=f"Cache {operation} failed for key '{key}': {reason}",
            component="cache",
            original_exception=original_exception,
```

```
        )

    def __str__(self):
        return f"[CacheException:{self.operation}] {self.message}"

# Repository-specific exceptions
class RepositoryException(InfrastructureException):
    """Base exception for repository layer errors."""

    def __init__(
        self,
        repository_name: str,
        operation: str,
        reason: str,
        original_exception: Optional[Exception] = None,
    ):
        self.repository_name = repository_name
        self.operation = operation
        self.reason = reason
        super().__init__(
            message=f"Repository '{repository_name}' {operation} failed: {reason}",
            component="repository",
            original_exception=original_exception,
        )

    def __str__(self):
        return f"[RepositoryException:{self.repository_name}.{self.operation}] {self.message}"

class DataIntegrityException(RepositoryException):
    """Raised when data integrity constraints are violated."""

    def __init__(
        self,
        constraint_name: str,
        details: str,
        original_exception: Optional[Exception] = None,
    ):
        self.constraint_name = constraint_name
        self.details = details
        super().__init__(
            repository_name="database",
            operation="constraint_validation",
            reason=f"Constraint '{constraint_name}' violated: {details}",
            original_exception=original_exception,
        )

    def __str__(self):
        return f"[DataIntegrityException:{self.constraint_name}] {self.message}"

class DataCorruptionException(RepositoryException):
    """Raised when data corruption is detected."""

    def __init__(self, entity_type: str, entity_id: str, corruption_details: str):
        self.entity_type = entity_type
        self.entity_id = entity_id
        self.corruption_details = corruption_details
        super().__init__(
            repository_name="database",
            operation="data_validation",
            reason=f"{entity_type} {entity_id} has corrupted data: {corruption_details}",
        )

    def __str__(self):
        return f"[DataCorruptionException:{self.entity_type}:{self.entity_id}] {self.message}"
```

14. Arquivo: interfaces.py

```
#./src/dev_platform/domain/user/interfaces.py
# -*- coding: utf-8 -*-
"""
Este módulo define a interface IUserRepository, que especifica os métodos necessários
para manipulação de usuários no repositório.
"""

from abc import ABC, abstractmethod
from typing import List, Optional
from dev_platform.domain.user.entities import User
from dev_platform.domain.user.value_objects import Email


class IUserRepository(ABC):
    @abstractmethod
    async def save(self, user: User) -> User:
        """Salva um usuário no repositório."""
        pass

    @abstractmethod
    async def find_by_email(self, email: Email) -> Optional[User]:
        """Busca um usuário pelo e-mail. Retorna o usuário ou None se não encontrado."""
        pass

    @abstractmethod
    async def find_all(self) -> List[User]:
        """Busca todos os usuários."""
        pass

    @abstractmethod
    async def find_by_id(self, user_id: int) -> Optional[User]:
        """Busca um usuário pelo ID. Retorna o usuário ou None se não encontrado."""
        pass

    @abstractmethod
    async def delete(self, user_id: int) -> bool:
        """Deleta um usuário pelo ID."""
        pass

    @abstractmethod
    async def find_by_name_contains(self, name_part: str) -> List[User]:
        """Busca usuários por uma parte do nome."""
        pass

    @abstractmethod
    async def count(self) -> int:
        """Conta o número total de usuários."""
        pass
```

15. Arquivo: logger.py

```
# src/dev_platform/application/ports/logger.py
# -*- coding: utf-8 -*-
"""
Este módulo define a interface ILogger para um logger estruturado,
permitindo a injeção de dependências e abstraindo a implementação específica de logging.
"""

from abc import ABC, abstractmethod
from typing import Optional


class ILogger(ABC):
    """Interface for a structured logger."""

    @abstractmethod
    def set_correlation_id(self, correlation_id: Optional[str] = None):
        """Set a correlation ID for tracking logs across requests."""
        raise NotImplementedError

    @abstractmethod
    def info(self, message: str, **kwargs):
        """Log an informational message."""
        raise NotImplementedError

    @abstractmethod
    def error(self, message: str, **kwargs):
        """Log an error message."""
        raise NotImplementedError

    @abstractmethod
    def warning(self, message: str, **kwargs):
        """Log a warning message."""
        raise NotImplementedError

    @abstractmethod
    def debug(self, message: str, **kwargs):
        """Log a debug message."""
        raise NotImplementedError

    @abstractmethod
    def critical(self, message: str, **kwargs):
        """Log a critical message."""
        raise NotImplementedError
```

16. Arquivo: main.py

```
# ./src/dev_platform/main.py
# -*- coding: utf-8 -*-
"""
Este módulo define a interface de linha de comando (CLI) para o DEV Platform,
permitindo a interação com os comandos relacionados a usuários.
"""

import click
# Importe user_cli do user_commands (renomeado para evitar conflito)
from dev_platform.interface.cli.user_commands import cli


# Cria um grupo Click principal
@click.group()
def main_cli():
    """CLI para o DEV Platform."""
    pass

# Adiciona os comandos de usuário como um subgrupo 'user'
main_cli.add_command(cli, name="user")

if __name__ == "__main__":
    main_cli()
```

17. Arquivo: mappers.py

```
# ./src/dev_platform/application/user/mappers.py
# -*- coding: utf-8 -*-
"""
Este módulo define os mapeadores para converter entre entidades User e seus Data Transfer Objects (DT
Os),
permitindo a transferência de dados entre camadas da aplicação.
"""

from dev_platform.domain.user.entities import User
from dev_platform.application.user.dtos import UserDTO, UserCreateDTO, UserUpdateDTO
from typing import List

def user_to_dto(user: User) -> UserDTO:
    return UserDTO(id=str(user.id), name=user.name.value, email=user.email.value)

def dto_to_user(dto: UserDTO) -> User:
    return User.create(name=dto.name, email=dto.email)

def create_dto_to_user(dto: UserCreateDTO) -> User:
    return User.create(name=dto.name, email=dto.email)

def update_dto_to_user(user_id: str, dto: UserUpdateDTO) -> User:
    # Supondo que User.create aceita id como argumento opcional
    return User.create(id=user_id, name=dto.name, email=dto.email)

def users_to_dtos(users: List[User]) -> List[UserDTO]:
    return [user_to_dto(user) for user in users]

# Para get, basta usar user_to_dto
# Para delete, normalmente só o id é necessário, não precisa de conversão extra
```

18. Arquivo: mkdocs.yml

```yaml
site_name: Dev Platform
site_description: Descrição do seu projeto
site_author: Seu Nome
site_url: https://exemplo.com/

repo_name: seu-usuario/seu-projeto
repo_url: https://github.com/seu-usuario/seu-projeto

theme:
  name: material
  features:
    - navigation.instant
    - navigation.tracking
    - navigation.expand
    - navigation.indexes
    - navigation.top
    - search.highlight
    - search.share
  palette:
    - media: "(prefers-color-scheme: light)"
      scheme: default
      primary: indigo
      accent: indigo
      toggle:
        icon: material/toggle-switch-off-outline
        name: Mudar para modo escuro
    - media: "(prefers-color-scheme: dark)"
      scheme: slate
      primary: indigo
      accent: indigo
      toggle:
        icon: material/toggle-switch
        name: Mudar para modo claro

markdown_extensions:
  - admonition
  - attr_list
  - def_list
  - footnotes
  - md_in_html
  - pymdownx.details
  - pymdownx.emoji:
      emoji_index: !!python/name:material.extensions.emoji.twemoji
      emoji_generator: !!python/name:material.extensions.emoji.to_svg
  - pymdownx.highlight:
      anchor_linenums: true
  - pymdownx.inlinehilite
  - pymdownx.snippets
  - pymdownx.superfences
  - pymdownx.tabbed:
      alternate_style: true
  - pymdownx.tasklist:
      custom_checkbox: true
  - tables
  - toc:
      permalink: true

plugins:
  - search
  - mkdocstrings:
      handlers:
        python:
          paths: [src]  # Onde buscar pelo código fonte
```

```
        options:
          docstring_style: google
          show_source: true
          show_submodules: true

nav:
  - Início: index.md
  - Guia do Usuário:
    - user-guide/getting-started.md
    - user-guide/advanced-usage.md
  - Documentação da API:
    - Visão Geral: api/index.md
    - Application: api/application.md
    - Domain: api/domain.md
    - Infrastructure: api/infrastructure.md
    - Interface: api/interface.md
    - Shared: api/shared.md
  - Desenvolvimento:
    - development/contributing.md
    - development/architecture.md
  - Changelog: changelog.md

extra:
  social:
    - icon: fontawesome/brands/github
      link: https://github.com/seu-usuario
    - icon: fontawesome/brands/twitter
      link: https://twitter.com/seu-usuario
    - icon: fontawesome/brands/linkedin
      link: https://linkedin.com/in/seu-usuario

copyright: Copyright &copy; 2025 Pereira Dev
```

19. Arquivo: models.py

```
# ./src/dev_platform/infrastructure/database/models.py
# -*- coding: utf-8 -*-
"""
Este módulo define os modelos de banco de dados usando SQLAlchemy,
permitindo a persistência de entidades do domínio em um banco de dados relacional.
"""

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base

Base = declarative_base()


class UserModel(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(100), nullable=False, unique=True)

    def __repr__(self):
        return f"<UserModel(id={self.id}, name='{self.name}', email='{self.email}')>"
```

20. Arquivo: mypy.ini

```
[mypy]
ignore_missing_imports = True
plugins = pydantic.mypy,sqlalchemy.ext.mypy.plugin
python_version = 3.11
```

21. Arquivo: poetry.toml

```
[virtualenvs]
in-project = true
create = true
```

22. Arquivo: ports.py

```
# ./src/dev_platform/application/user/ports.py
# -*- coding: utf-8 -*-
"""
Este módulo define as interfaces para os repositórios e serviços relacionados à entidade User.
"""

from abc import ABC, abstractmethod

from dev_platform.domain.user.interfaces import IUserRepository


class UnitOfWork(ABC):
    """
    Interface for the Unit of Work pattern.
    """
    users: IUserRepository

    @abstractmethod
    async def __aenter__(self):
        """Enter the context of the unit of work."""
        raise NotImplementedError

    @abstractmethod
    async def __aexit__(self, exc_type, exc_val, exc_tb):
        """Exit the context of the unit of work."""
        raise NotImplementedError

    @abstractmethod
    async def commit(self):
        """Commit the changes made during the unit of work."""
        raise NotImplementedError
```

23. Arquivo: pyproject.toml

```
# ./pyproject.toml
[project]
name = "dev-platform"
version = "0.1.0"
description = ""
authors = [
    {name = "Sergio Pereira",email = "sergiopereira.br@hotmail.com"}
]
readme = "README.md"
requires-python = ">=3.11,<4.0"
dependencies = [
    "pymysql (>=1.1.1,<2.0.0)",
    "aiomysql (>=0.2.0,<0.3.0)",
    "typer (>=0.15.4,<0.16.0)",
    "pydantic[email] (>=2.11.7,<3.0.0)",
    "loguru (>=0.7.3,<0.8.0)",
    "uuid (>=1.30,<2.0)",
    "alembic (>=1.16.1,<2.0.0)",
    "sqlalchemy (>=2.0.41,<3.0.0)",
    "python-dotenv (>=1.1.0,<2.0.0)",
    "pandas (>=2.3.0,<3.0.0)",
    "cryptography (>=45.0.4,<46.0.0)",
    "pydantic-settings (>=2.9.1,<3.0.0)",
]

[tool.poetry]
packages = [{include = "dev_platform", from = "src"}]

[tool.poetry.group.dev.dependencies]
black = "^23.0"
flake8 = "^6.0"
taskipy = "^1.14.1"
reportlab = "^4.4.1"
chardet = "^5.2.0"
pypandoc = "^1.15"
mypy = "^1.16.0"
pytest-asyncio = "^1.0.0"
types-reportlab = "^4.4.1.20250602"
matplotlib = "^3.10.3"
numpy = "^2.3.0"
python-dateutil = "^2.9.0.post0"

[tool.poetry.group.docs.dependencies]
mkdocs = "^1.6.1"
pymdown-extensions = "^10.15"
mkdocstrings = "^0.29.1"
mkdocstrings-python = "^1.16.10"
sphinx = "^6.0"
sphinx-rtd-theme = "^1.0"
mkdocs-material = "^9.6.14"


[tool.poetry.group.test.dependencies]
pytest = "^8.3.5"

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"

# Configuração opcional para definir o script principal
[tool.poetry.scripts]
dev-platform = "dev_platform.main:main"  # Isso cria um comando executável
docs-serve = "mkdocs:serve"  # Iniciar servidor de documentação local
```

```
docs-build = "mkdocs:build"  # Construir a documentação estática

[tool.taskipy.tasks]
clean = "pwsh -Command ./scripts/tools/del_folderes.ps1 './src' '__pycache__'"
stack = "pwsh -Command ./scripts/tools/stack_files.ps1 './src' 'py'"
list = "poetry run python ./src/dev_platform/main.py user list-users"
files2pdf = "poetry run python ./scripts/tools/pdf_generator.py ./src/stake_file --title \"Arquivos do Projet
o DEV Platform\" -v"
analise_md2word = "poetry run python ./scripts/tools/md_to_word.py ./docs/análise_26.md" # Converte
o arquivo Markdown em Word - Melhor no WSL Ubuntu 20.04
onboarding_md2word = "poetry run python ./scripts/tools/md_to_word.py ./docs/onboarding_26.md" #
Converte o arquivo Markdown em Word - Melhor no WSL Ubuntu 20.04
analisa_log = "poetry run python ./scripts/tools/log_analyzer_mvp.py --log_folder_path ./scripts/requisitos
_log"
compilado = "poetry run python ./scripts/tools/pdf_generator.py ./src/stake_file --title \"Arquivos do Proj
eto DEV Platform\" -v"

[tool.mypy]
python_version = "3.11"
plugins = "pydantic.mypy"
```

24. Arquivo: README.md

<Arquivo vazio ou sem conteúdo legível>

25. Arquivo: repositories.py

```
# ./src/dev_platform/infrastructure/database/repositories.py
# -*- coding: utf-8 -*-
"""
This module implements the SQLAlchemy repository for the User entity,
providing methods for CRUD operations and queries.
"""

from typing import List, Optional
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.future import select
from sqlalchemy import delete, func
from sqlalchemy.exc import SQLAlchemyError, IntegrityError
from dev_platform.domain.user.interfaces import IUserRepository
from dev_platform.domain.user.entities import User
from dev_platform.domain.user.value_objects import UserName, Email
from dev_platform.application.ports.logger import ILogger
from dev_platform.domain.exceptions import (
    DatabaseException,
)
from dev_platform.domain.user.user_exceptions import (
    UserAlreadyExistsException,
    UserNotFoundException
)
from dev_platform.infrastructure.database.models import UserModel


class SQLUserRepository(IUserRepository):
    """SQLAlchemy implementation of the IUserRepository interface."""
    def __init__(self, session: AsyncSession, logger: ILogger):
        self._session = session
        self._logger: ILogger = logger

    def _convert_to_domain_user(self, db_user: UserModel) -> User:
        """Convert database model to domain entity."""
        try:
            return User(
                id=db_user.id, name=UserName(db_user.name), email=Email(db_user.email)
            )
        except ValueError as e:
            self._logger.error(
                f"Data conversion error for user {db_user.id}: {str(e)}"
            )
            # This should not happen if database constraints are properly set
            raise DatabaseException(
                operation="data_conversion",
                reason=f"Invalid data in database: {str(e)}",
                original_exception=e,
            )

    async def save(self, user: User) -> User:
        """Save a user to the database."""
        try:
            if user.id is None:
                # Create new user
                db_user = UserModel(name=user.name.value, email=user.email.value)
                self._session.add(db_user)
                await self._session.flush()

                # Return user with the generated ID
                return User(id=db_user.id, name=user.name, email=user.email)
            else:
                # Update existing user
                result = await self._session.execute(
```

```
                select(UserModel).where(UserModel.id == user.id)
            )
            db_user = result.scalars().first()

            if not db_user:
                raise UserNotFoundException(str(user.id))

            db_user.name = user.name.value
            db_user.email = user.email.value
            await self._session.flush()

            return User(id=db_user.id, name=user.name, email=user.email)

        except UserNotFoundException:
            # Re-raise domain exceptions as-is
            raise
        except UserAlreadyExistsException:
            # Re-raise domain exceptions as-is
            raise
        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in save_user",
                extra={
                    "operation": "save_user",
                    "user_id": user.id,
                    "name": user.name.value,
                    "email": user.email.value,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(
                operation="save_user",
                error=e,
                user_id=user.id,
                name=user.name.value,
                email=user.email.value
            )

    async def find_by_email(self, email: str) -> Optional[User]:
        """Find a user by email address."""
        try:
            result = await self._session.execute(
                select(UserModel).where(UserModel.email == email)
            )
            db_user = result.scalars().first()

            if db_user:
                return self._convert_to_domain_user(db_user)
            return None

        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in find_by_email",
                extra={
                    "operation": "find_by_email",
                    "email": email,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(operation="find_by_email", error=e, email=
email)
        return None

    async def find_all(self) -> List[User]:
        """Find all users in the database."""
        try:
```

```
            result = await self._session.execute(select(UserModel))
            db_users = result.scalars().all()

            return [self._convert_to_domain_user(db_user) for db_user in db_users]

        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in find_all_users",
                extra={
                    "operation": "find_all_users",
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(operation="find_all_users", error=e)
        return []

    async def find_by_id(self, user_id: int) -> Optional[User]:
        """Find a user by ID."""
        try:
            result = await self._session.execute(
                select(UserModel).where(UserModel.id == user_id)
            )
            db_user = result.scalars().first()

            if db_user:
                return self._convert_to_domain_user(db_user)
            return None

        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in find_by_id",
                extra={
                    "operation": "find_by_id",
                    "user_id": user_id,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(
                operation="find_by_id", error=e, user_id=user_id
            )
        return None

    async def find_by_ids(self, user_ids: List[int]) -> List[User]:
        """Find users by a list of IDs."""
        try:
            result = await self._session.execute(
                select(UserModel).where(UserModel.id.in_(user_ids))
            )
            return [self._convert_to_domain_user(u) for u in result.scalars().all()]
        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in find_by_ids",
                extra={
                    "operation": "find_by_ids",
                    "user_ids": user_ids,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(operation="find_by_ids", error=e, user_ids=
user_ids)
        return []

    async def delete(self, user_id: int) -> bool:
        """Delete a user by ID."""
        try:
            # First check if user exists
```

```
            existing_user = await self.find_by_id(user_id)
            if not existing_user:
                raise UserNotFoundException(str(user_id))

            # Perform deletion
            result = await self._session.execute(
                delete(UserModel).where(UserModel.id == user_id)
            )

            success = result.rowcount > 0
            if success:
                await self._session.flush()

            return success

        except UserNotFoundException:
            # Re-raise domain exceptions as-is
            raise
        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in delete_user",
                extra={
                    "operation": "delete_user",
                    "user_id": user_id,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(
                operation="delete_user", error=e, user_id=user_id
            )
        return False

    async def find_by_name_contains(self, name_part: str) -> List[User]:
        """Find users whose name contains the given string."""
        try:
            result = await self._session.execute(
                select(UserModel).where(UserModel.name.contains(name_part))
            )
            db_users = result.scalars().all()

            return [self._convert_to_domain_user(db_user) for db_user in db_users]

        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in find_by_name_contains",
                extra={
                    "operation": "find_by_name_contains",
                    "name_part": name_part,
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(
                operation="find_by_name_contains", error=e, name_part=name_part
            )
        return []

    async def count(self) -> int:  # Nota
        """Count total number of users."""
        try:
            result = await self._session.execute(select(func.count(UserModel.id)))
            count = result.scalar()
            return count if count is not None else 0

        except SQLAlchemyError as e:
            self._logger.error(
                "SQLAlchemy error in count_users",
```

```
                extra={
                    "operation": "count_users",
                    "error": str(e)
                }
            )
            RepositoryExceptionHandler.handle_sqlalchemy_error(operation="count_users", error=e)

        return 0  # Nota de teste: Retornar 0 se a contagem falhar, o que é mais intuitivo do que retornar No
ne.


class RepositoryExceptionHandler:
    """Utility class for handling repository exceptions consistently."""

    @staticmethod
    def handle_sqlalchemy_error(operation: str, error: SQLAlchemyError, **context):
        """Handle SQLAlchemy specific errors."""
        if isinstance(error, IntegrityError):
            if (
                "email" in str(error.orig).lower()
                and "unique" in str(error.orig).lower()
            ):
                email = context.get("email", "unknown")
                raise UserAlreadyExistsException(email)

        context_str = ", ".join([f"{k}={v}" for k, v in context.items()])
        error_msg = f"{operation} failed"
        if context_str:
            error_msg += f" ({context_str})"

        raise DatabaseException(
            operation=operation, reason=str(error), original_exception=error
        )

    @staticmethod
    def handle_generic_error(operation: str, error: Exception, **context):
        """Handle generic errors."""
        context_str = ", ".join([f"{k}={v}" for k, v in context.items()])
        error_msg = f"{operation} failed"
        if context_str:
            error_msg += f" ({context_str})"

        raise DatabaseException(
            operation=operation, reason=str(error), original_exception=error
        )
```

26. Arquivo: services.py

```
# ./src/dev_platform/domain/user/services.py
# -*- coding: utf-8 -*-
"""
Este módulo define os serviços de domínio relacionados à entidade User.
Os serviços são responsáveis por validações complexas, regras de negócio e operações
com usuários.
"""

from abc import ABC, abstractmethod
from typing import List, Dict, Optional, Set
import re
from datetime import datetime
from dev_platform.domain.user.interfaces import IUserRepository
from dev_platform.domain.user.entities import User
from dev_platform.domain.exceptions import DatabaseException
from dev_platform.domain.user.user_exceptions import (
    UserValidationException,
    UserAlreadyExistsException,
    UserNotFoundException,
    EmailDomainNotAllowedException,
)

# --- Regras de validação devem ser extraídas para um módulo próprio (ex: validation_rules.py) ---
# Aqui mantemos apenas a interface base para uso no domínio.

class ValidationRule(ABC):
    """Base class for validation rules."""

    @abstractmethod
    async def validate(self, user: User) -> Optional[str]:
        """
        Validate user according to this rule.
        Returns None if valid, error message if invalid.
        """
        pass

    @property
    @abstractmethod
    def rule_name(self) -> str:
        pass

# --- Serviço de domínio focado apenas na lógica de negócio ---

class UserUniquenessService:
    """Service focused on uniqueness validation."""

    def __init__(self, user_repository: IUserRepository):
        self._repository = user_repository

    async def ensure_email_is_unique(
        self, email: str, exclude_user_id: Optional[int] = None
    ) -> None:
        existing_user = await self._repository.find_by_email(email)
        if existing_user and (
            exclude_user_id is None or existing_user.id != exclude_user_id
        ):
            raise UserAlreadyExistsException(email)

class UserDomainService:
    """
    Service for complex user domain validations and business rules.
    Recebe explicitamente as regras de validação a serem aplicadas.
    """
```

```
def __init__(self, validation_rules: List[ValidationRule]):
    self._validation_rules = validation_rules

def add_validation_rule(self, rule: ValidationRule):
    """Add a custom validation rule."""
    self._validation_rules.append(rule)

def remove_validation_rule(self, rule_name: str):
    """Remove a validation rule by name."""
    self._validation_rules = [
        rule for rule in self._validation_rules if rule.rule_name != rule_name
    ]

async def validate_business_rules(self, user: User) -> None:
    """
    Validate all business rules for a user.
    Raises UserValidationException if any rule fails.
    """
    validation_errors = {}

    # Run all validation rules
    for rule in self._validation_rules:
        try:
            error_message = await rule.validate(user)
            if error_message:
                validation_errors[rule.rule_name] = error_message
        except Exception as e:
            validation_errors[rule.rule_name] = f"Validation rule failed: {str(e)}"

    # If there are validation errors, raise exception
    if validation_errors:
        raise UserValidationException(validation_errors)

async def validate_user_update(self, user_id: int, updated_user: User) -> None:
    """
    Validate user update, checking uniqueness only if email changed.
    """
    validation_errors = {}

    # Get current user
    current_user = await self._repository.find_by_id(user_id)
    if not current_user:
        raise UserNotFoundException(str(user_id))

    # Check email uniqueness only if email changed
    if current_user.email.value != updated_user.email.value:
        try:
            await self._uniqueness_service.ensure_email_is_unique(updated_user.email.value)
        except UserAlreadyExistsException as e:
            validation_errors["email"] = e.message

    # Run validation rules
    for rule in self._validation_rules:
        try:
            error_message = await rule.validate(updated_user)
            if error_message:
                validation_errors[rule.rule_name] = error_message
        except Exception as e:
            validation_errors[rule.rule_name] = f"Validation rule failed: {str(e)}"

    if validation_errors:
        raise UserValidationException(validation_errors)

def get_validation_summary(self) -> Dict[str, str]:
    """Get summary of all active validation rules."""
```

```
        return {
            rule.rule_name: rule.__class__.__doc__ or "No description available"
            for rule in self._validation_rules
        }

    async def validate_user_creation_constraints(self, user: User) -> None:
        """
        Validate constraints specific to user creation.
        (Exemplo: limite de usuários, regras de negócio específicas)
        """
        validation_errors = {}

        try:
            current_count = await self._repository.count()
            if current_count >= 10000:  # Exemplo de limite
                validation_errors["system_limit"] = "Maximum number of users reached"
        except Exception as e:
            validation_errors["system_check"] = f"Unable to verify system constraints: {str(e)}"

        if validation_errors:
            raise UserValidationException(validation_errors)

    async def validate_business_domain_rules(
        self, user: User, domain_whitelist: Optional[List[str]] = None
    ) -> None:
        """
        Validate business-specific domain rules.
        """
        if domain_whitelist:
            email_domain = user.email.value.split("@")[1].lower()
            if email_domain not in [d.lower() for d in domain_whitelist]:
                raise EmailDomainNotAllowedException(
                    user.email.value, email_domain, domain_whitelist
                )

class UserAnalyticsService:
    """Service for user analytics and reporting."""

    def __init__(self, user_repository: IUserRepository):
        self._repository = user_repository

    async def get_user_statistics(self) -> Dict[str, int]:
        """Get basic user statistics."""
        try:
            total_users = await self._repository.count()
            return {
                "total_users": total_users,
            }
        except Exception as e:
            raise DatabaseException("count", str(e), e)

    async def find_users_by_domain(self, domain: str) -> List[User]:
        """Find all users with emails from a specific domain."""
        try:
            all_users = await self._repository.find_all()
            return [
                user
                for user in all_users
                if user.email.value.split("@")[1].lower() == domain.lower()
            ]
        except Exception as e:
            raise DatabaseException("find_all", str(e), e)
```

27. Arquivo: session.py

```
# ./src/dev_platform/infrastructure/database/session.py
# -*- coding: utf-8 -*-
"""
Este módulo define o gerenciador de sessões de banco de dados,
centralizando a criação e gerenciamento de sessões síncronas e assíncronas usando SQLAlchemy.
Ele permite o uso de context managers para garantir o commit automático e rollback em caso de exceçõe
s.
"""

from contextlib import asynccontextmanager
from typing import AsyncGenerator, Optional
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
from sqlalchemy.engine import Engine
from sqlalchemy.ext.asyncio import create_async_engine, AsyncEngine, AsyncSession, async_sessionmaker
from dev_platform.infrastructure.config import ConfigurationFacade


class DatabaseSessionManager:
    """
    Gerenciador centralizado de sessões de banco de dados.

    ⚠      Commit automático no context manager
    O método get_async_session realiza automaticamente um await session.commit() ao sair do bloco asyn
c with.

    Se ocorrer uma exceção dentro do bloco, será feito await session.rollback() antes de propagar o erro.

    Atenção:
    Esse comportamento pode surpreender desenvolvedores que esperam controlar o commit manualmen
te.
    Se você precisa de controle total sobre commit/rollback, utilize a factory de sessão diretamente.

    Exemplo de uso:
    async with db_manager.get_async_session() as session:
        # Todas as operações dentro deste bloco serão automaticamente commitadas ao final,
        # ou rollback em caso de exceção.
        ...

    Resumo:
    O commit é automático ao sair do bloco.
    O rollback é automático em caso de exceção.
    Para controle manual, crie a sessão diretamente via factory."""

    def __init__(self, config: Optional[ConfigurationFacade] = None):
        self._async_engine: Optional[AsyncEngine] = None
        self._sync_engine: Optional[Engine] = None
        self._async_session_factory: Optional[async_sessionmaker] = None
        self._sync_session_factory: Optional[sessionmaker] = None
        self.config = config
        self._initialize_engines()
        self.config = ConfigurationFacade()

    def _initialize_engines(self):
        """Inicializa os engines síncronos e assíncronos."""
        # Configurações do pool
        pool_config = {
            "pool_size": int(self.config.get("database_pool_size", 5)),
            "max_overflow": int(self.config.get("database_max_overflow", 10)),
            "pool_pre_ping": self.config.get("database_pool_pre_ping", True),
        }

        # Engine assíncrono
```

```
        async_url = self.config.get("database_url")
        self._async_engine = create_async_engine(
            async_url, echo = self.config.get("database_echo", False), **pool_config
        )

        # Session factory assíncrona
        self._async_session_factory = async_sessionmaker(
            bind = self._async_engine, class_=AsyncSession, expire_on_commit=False
        )

        # Engine síncrono (se necessário para migrações ou outras operações)
        if not async_url.startswith(
            "sqlite+aiosqlite"
        ):  # SQLite não precisa de engine síncrono separado
            sync_url = self.config.get("database_url")
            self._sync_engine = create_engine(
                sync_url, echo = self.config.get("database_echo", False), **pool_config
            )

            self._sync_session_factory = sessionmaker(
                bind = self._sync_engine, autocommit=False, autoflush=False
            )

    @asynccontextmanager
    async def get_async_session(self) -> AsyncGenerator[AsyncSession, None]:
        """
        Context manager para sessões assíncronas de banco de dados.

        ⚠️      Commit automático: Ao sair do bloco async with, será feito await session.commit().
        Em caso de exceção, será feito await session.rollback() automaticamente.

        Se precisar de controle manual sobre commit/rollback, utilize a factory de sessão diretamente.

        Exemplo:
            async with db_manager.get_async_session() as session:
                # Operações...
        """
        if self._async_session_factory is None:
            raise RuntimeError("Async session factory is not initialized")
        async with self._async_session_factory() as session:
            try:
                yield session
                await session.commit()
            except Exception:
                await session.rollback()
                raise

    def get_sync_session(self) -> Session:
        """Obtém uma sessão síncrona (para migrações, etc.)."""
        if not self._sync_session_factory:
            raise RuntimeError("Sync session not available for this database type")
        return self._sync_session_factory()

    async def close_async_engine(self):
        """Fecha o engine assíncrono."""
        if self._async_engine:
            await self._async_engine.dispose()

    def close_sync_engine(self):
        """Fecha o engine síncrono."""
        if self._sync_engine:
            self._sync_engine.dispose()

    @property
    def async_engine(self) -> AsyncEngine:
        """Propriedade para acessar o engine assíncrono."""
```

```
        return self._async_engine

    @property
    def sync_engine(self) -> Engine:
        """Propriedade para acessar o engine síncrono."""
        return self._sync_engine


# Instância global do gerenciador de sessões
config = ConfigurationFacade()
db_manager = DatabaseSessionManager(config=config)

# Funções de conveniência para compatibilidade
async def get_async_session():
    """Função de conveniência para obter sessão assíncrona.
    Garante que o gerenciador de sessões assíncronas esteja inicializado
    e faz await session.commit() automaticamente ao sair do bloco."""
    if db_manager._async_session_factory is None:
        raise RuntimeError("Async session factory is not initialized")
    async with db_manager.get_async_session() as session:
        yield session

def get_sync_session():
    """Função de conveniência para obter sessão síncrona."""
    return db_manager.get_sync_session()

# Aliases para compatibilidade com código existente
if db_manager._async_session_factory:
    AsyncSessionLocal = db_manager._async_session_factory

if db_manager._sync_session_factory:
    SessionLocal = db_manager._sync_session_factory
```

28. Arquivo: settings.json

```
{
    "python.terminal.activateEnvironment": true,
    "python.defaultInterpreterPath": "${workspaceFolder}/dev_platform/.venv/Scripts/python.exe",
    "python.analysis.extraPaths": [
        "${workspaceFolder}/dev_platform/src"
    ]
}
```

29. Arquivo: structured_logger.py

```
# src/dev_platform/infrastructure/logging/structured_logger.py
# -*- coding: utf-8 -*-
"""
Este módulo implementa um logger estruturado usando Loguru,
com suporte a níveis dinâmicos e correlação de logs.
Ele permite a configuração de handlers para diferentes níveis de log
e formatos, incluindo JSON para fácil integração com sistemas de monitoramento.
"""

from typing import Optional
import os
from uuid import uuid4
from loguru import logger
# from dev_platform.infrastructure.config import CONFIG
from dev_platform.application.ports.logger import ILogger


class StructuredLogger(ILogger):
    """Logger estruturado usando Loguru com suporte a níveis dinâmicos e correlação de logs."""

    def __init__(self, name: str = "DEV Platform", CONFIG__: Optional[object] = None):
        self._name = name
        self._CONFIG__ = CONFIG__ or {}
        self._configure_logger()

    def _configure_logger(self):
        """Configura o logger com base no ambiente e adiciona handlers."""
        # Remover handlers padrão do Loguru
        logger.remove()

        # Obter nível de log com base no ambiente
        environment = self._CONFIG__.get("environment", "production")
        log_level = self._CONFIG__.get("logging_level", "INFO").upper()
        log_levels = {"development": "DEBUG", "test": "DEBUG", "production": "INFO"}
        default_level = log_levels.get(environment, "INFO")
        final_level = (
            log_level
            if log_level in ["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"]
            else default_level
        )

        # Configurar handler para console (JSON, todos os níveis)
        logger.add(
            sink="sys.stdout",
            level=final_level,
            format="{time:YYYY-MM-DD HH:mm:ss.SSS} | {level} | {message} | {extra}",
            serialize=True,  # Formato JSON
        )

        # Configurar handler para arquivo (apenas ERROR, com rotação)
        if not os.path.exists("logs"):
            os.makedirs("logs")
        logger.add(
            sink=f"logs/{self._name}_{{time:YYYY-MM-DD}}.log",
            level="ERROR",
            rotation="10 MB",
            retention="5 days",
            compression="zip",
            enqueue=True,  # Assíncrono
            serialize=True,  # <-- Adicione esta linha para JSON
        )

    def set_correlation_id(self, correlation_id: Optional[str] = None):
```

```
    """Define um ID de correlação para rastreamento."""
    self._logger = logger.bind(correlation_id=correlation_id or str(uuid4()))

def info(self, message: str, **kwargs):
    """Registra uma mensagem de nível INFO."""
    (self._logger if hasattr(self, "_logger") else logger).bind(**kwargs).info(message)

def error(self, message: str, **kwargs):
    """Registra uma mensagem de nível ERROR."""
    (self._logger if hasattr(self, "_logger") else logger).bind(**kwargs).error(message)

def warning(self, message: str, **kwargs):
    """Registra uma mensagem de nível WARNING."""
    (self._logger if hasattr(self, "_logger") else logger).bind(**kwargs).warning(message)

def debug(self, message: str, **kwargs):
    """Registra uma mensagem de nível DEBUG."""
    (self._logger if hasattr(self, "_logger") else logger).bind(**kwargs).debug(message)

def critical(self, message: str, **kwargs):
    """Registra uma mensagem de nível CRITICAL."""
    (self._logger if hasattr(self, "_logger") else logger).bind(**kwargs).critical(message)

# NOVO MÉTODO PARA SHUTDOWN GRACIOSO DO LOGGER
@staticmethod
def shutdown():
    """
    Garante que todas as mensagens enfileiradas pelo Loguru sejam processadas
    e que os handlers sejam removidos. Isso é crucial para limpar recursos
    assíncronos do logger antes que o loop de eventos feche.
    """
    logger.shutdown()  # Processa todas as mensagens enfileiradas
    logger.remove()  # Remove todos os handlers para evitar vazamentos de recursos
```

30. Arquivo: tree.txt

```
./tree.txt
# Directory structure for the DEV Platform project
# Esse arquivo contém a estrutura de diretórios do projeto DEV Platform
# e é utilizado para gerar a documentação do projeto.
# A estrutura é gerada automaticamente pelo comando `tree` no terminal.

dev_platform
|   .env
|   .env.development
|   .env.production
|   .env.test
|   .gitignore
|   alembic.ini
|   config.production.json
|   mkdocs.yml
|   mypy.ini
|   mypy.txt
|   poetry.lock
|   poetry.toml
|   pyproject.toml
|   README.md
|   sys.stdout
|   tree.txt
|   tree3.txt
|
+---.venv
|   |   pyvenv.cfg
|   |   Scripts
|   |   |   activate
|   |   |   activate.bat
|   |   |   deactivate.bat
|   |   |   pip.exe
|   |   |   python.exe
|   |   |   pythonw.exe
|   |   |   ...
|   |   |
|   \---Lib
|       |   site-packages
|       |   |   ...
|       |   |
|       \---...
+---docs
|
+---logs
|       DEV Platform_2025-06-01.log
|       DEV Platform_2025-06-02.log
|       DEV Platform_2025-06-03.log
|       DEV Platform_2025-06-05.log
|       DEV Platform_2025-06-06.log
|       DEV Platform_2025-06-07.log
|       DEV Platform_2025-06-08.log
|       DEV Platform_2025-06-09.log
|       DEV Platform_2025-06-11.log
|       DEV Platform_2025-06-12.log
|       DEV Platform_2025-06-13.log
|       DEV Platform_2025-06-14.log
|       DEV Platform_2025-06-15.log
|       DEV Platform_2025-06-17.log
|
+---migrations
|   |   env.py
|   |   README
|   |   script.py.mako
```

```
|  |
|  \---versions
|
+---scripts
|   +---log
|  |       amostra_log.pdf
|  |       conceito.pdf
|  |
|  \---tools
|           cria_pastas.ps1
|           cria_pastas_arquivo.ps1
|           del_folderes.ps1
|           log_analyzer_mvp.py
|           md_to_word.py
|           pdf_generator.py
|           stack_files.ps1
|           tree.ps1
|
+---tests
|       test_db.py
|
\---src
    +---dev_platform
    |  |  main.py
    |  |
    |  +---application
    |  |  +---ports
    |  |  |  |  logger.py
    |  |  |  |
    |  |  |  \---__pycache__
    |  |  |          logger.cpython-311.pyc
    |  |  |
    |  |  \---user
    |  |      |  dtos.py
    |  |      |  mappers.py
    |  |      |  ports.py
    |  |      |  use_cases.py
    |  |      |
    |  |      \---__pycache__
    |  |              dtos.cpython-311.pyc
    |  |              ports.cpython-311.pyc
    |  |              use_cases.cpython-311.pyc
    |  |
    |  +---domain
    |  |  |  exceptions.py
    |  |  |  validation_rules.py
    |  |  |
    |  |  +---user
    |  |  |  |  entities.py
    |  |  |  |  interfaces.py
    |  |  |  |  services.py
    |  |  |  |  user_exceptions.py
    |  |  |  |  value_objects.py
    |  |  |  |
    |  |  |  \---__pycache__
    |  |  |          entities.cpython-311.pyc
    |  |  |          interfaces.cpython-311.pyc
    |  |  |          services.cpython-311.pyc
    |  |  |          user_exceptions.cpython-311.pyc
    |  |  |          value_objects.cpython-311.pyc
    |  |  |
    |  |  \---__pycache__
    |  |          exceptions.cpython-311.pyc
    |  |          validation_rules.cpython-311.pyc
    |  |
    |  +---infrastructure
```

```
|  |  |  composition_root.py
|  |  |  config.py
|  |  |
|  |  +---database
|  |  |  |  models.py
|  |  |  |  repositories.py
|  |  |  |  session.py
|  |  |  |  unit_of_work.py
|  |  |  |
|  |  |  \---__pycache__
|  |  |          models.cpython-311.pyc
|  |  |          repositories.cpython-311.pyc
|  |  |          session.cpython-311.pyc
|  |  |          unit_of_work.cpython-311.pyc
|  |  |
|  |  +---logging
|  |  |  |  structured_logger.py
|  |  |  |
|  |  |  \---__pycache__
|  |  |          structured_logger.cpython-311.pyc
|  |  |
|  |  \---__pycache__
|  |          composition_root.cpython-311.pyc
|  |          config.cpython-311.pyc
|  |
|  \---interface
|      \---cli
|          |  user_commands.py
|          |
|          \---__pycache__
|                  user_commands.cpython-311.pyc
|
\---stake_file
        .env
        .env.development
        .env.production
        .env.test
        .gitignore
        alembic.ini
        composition_root.py
        config.production.json
        config.py
        dtos.py
        entities.py
        env.py
        exceptions.py
        interfaces.py
        logger.py
        main.py
        mappers.py
        mkdocs.yml
        models.py
        mypy.ini
        poetry.toml
        ports.py
        pyproject.toml
        README.md
        repositories.py
        services.py
        session.py
        settings.json
        structured_logger.py
        tree.txt
        unit_of_work.py
        user_commands.py
        user_exceptions.py
```

use_cases.py
validation_rules.py
value_objects.py

31. Arquivo: unit_of_work.py

```
# ./src/dev_platform/infrastructure/database/unit_of_work.py
# -*- coding: utf-8 -*-
"""
Este módulo implementa o padrão Unit of Work para gerenciar transações de banco de dados
e repositórios de usuários usando SQLAlchemy.
"""

from typing import Optional
from contextlib import AbstractAsyncContextManager

from sqlalchemy.ext.asyncio import AsyncSession

from dev_platform.infrastructure.config import ConfigurationFacade
from dev_platform.application.user.ports import UnitOfWork
from dev_platform.application.ports.logger import ILogger
from dev_platform.infrastructure.logging.structured_logger import StructuredLogger
from dev_platform.domain.user.interfaces import IUserRepository
from dev_platform.infrastructure.database.session import db_manager
from dev_platform.infrastructure.database.repositories import SQLUserRepository


class SQLUnitOfWork(UnitOfWork):
    def __init__(self, logger: Optional[ILogger] = StructuredLogger(CONFIG__=ConfigurationFacade())):
        self._session_context: Optional[AbstractAsyncContextManager[AsyncSession]] = None # Gerenciador
 de contexto para a sessão assíncrona
        self._logger: ILogger = logger or StructuredLogger(CONFIG__=ConfigurationFacade())
        self._user_repository: Optional[IUserRepository] = None
        self._session: Optional[AsyncSession] = None

    @property
    def user_repository(self) -> Optional[IUserRepository]:
        return self._user_repository

    async def __aenter__(self):
        # Usar o gerenciador de sessões
        self._session_context = db_manager.get_async_session()
        self._session = await self._session_context.__aenter__()
        self._user_repository = SQLUserRepository(self._session, logger=self._logger)
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        if not self._session:
            return
        try:
            if exc_type is None:
                await self._session.commit()
            else:
                await self._session.rollback()
        except Exception as e:
            self._logger.error(f"Error in transaction cleanup: {e}")
            try:
                await self._session.rollback()
            except:
                pass
        finally:
            try:
                await self._session.close()
                await self._session_context.__aexit__(exc_type, exc_val, exc_tb)
                self._session = None
                self._user_repository = None
            except Exception as e:
                self._logger.error(f"Error closing session: {e}")
```

```
async def commit(self):
    if self._session:
        await self._session.commit()

async def rollback(self):
    if self._session:
        await self._session.rollback()
```

32. Arquivo: use_cases.py

```
# ./src/dev_platform/application/user/use_cases.py
# -*- coding: utf-8 -*-
"""
Este módulo define os casos de uso para a entidade User,
encapsulando a lógica de negócios e as interações com os repositórios.
"""

from typing import List

from dev_platform.application.user.dtos import UserCreateDTO, UserUpdateDTO, UserDTO
from dev_platform.domain.user.entities import User
from dev_platform.application.ports.logger import ILogger
from dev_platform.application.user.ports import UnitOfWork
from dev_platform.domain.exceptions import DatabaseException
from dev_platform.domain.user.user_exceptions import (
    UserValidationException,
    UserAlreadyExistsException,
    UserNotFoundException,
    EmailDomainNotAllowedException,
)
from dev_platform.domain.user.services import UserDomainService, UserUniquenessService
from dev_platform.domain.validation_rules import (
    EmailFormatAdvancedValidationRule,
    NameContentValidationRule,
    EmailDomainValidationRule,
    BusinessHoursValidationRule,
)

# Helper function for entity to DTO conversion
def user_to_dto(user: User) -> UserDTO:
    return UserDTO(
        id=str(user.id),
        name=user.name.value if hasattr(user.name, "value") else user.name,
        email=user.email.value if hasattr(user.email, "value") else user.email,
    )

class BaseUseCase:
    """Base class for all use cases, providing access to Unit of Work and logger."""
    def __init__(self, uow: UnitOfWork, logger: ILogger):
        self._uow = uow
        self._logger = logger

class CreateUserUseCase(BaseUseCase):
    """Use case for creating a new user."""
    def __init__(
        self,
        uow: UnitOfWork,
        logger: ILogger,
        domain_service: UserDomainService,
    ):
        super().__init__(uow, logger)
        self._domain_service = domain_service

    async def execute(self, dto: UserCreateDTO) -> UserDTO:
        async with self._uow:
            self._logger.set_correlation_id()
            self._logger.info("Starting user creation", name=dto.name, email=dto.email)
            try:
                # 1. Caso de uso orquestra a verificação de unicidade PRIMEIRO
                existing_user = await self._uow.user_repository.find_by_email(dto.email)
                if existing_user:
                    raise UserAlreadyExistsException(dto.email)
```

```
            # 2. Cria a entidade de domínio
            user = User.create(name=dto.name, email=dto.email)

            # 3. Executa apenas as regras de negócio puras do domínio
            await self._domain_service.validate_business_rules(user)
            self._logger.info("User validation passed", email=dto.email)

            # 4. Salva o usuário
            saved_user = await self._uow.user_repository.save(user)
            await self._uow.commit()
            self._logger.info(
                "User created successfully",
                user_id=saved_user.id,
                name=saved_user.name.value if hasattr(saved_user.name, "value") else saved_user.name,
                email=saved_user.email.value if hasattr(saved_user.email, "value") else saved_user.email,
            )
            return user_to_dto(saved_user)
        except UserValidationException as e:
            self._logger.error(
                "User validation failed",
                email=dto.email,
                validation_errors=e.validation_errors,
            )
            raise
        except UserAlreadyExistsException as e:
            self._logger.warning(
                "Attempted to create duplicate user", email=dto.email
            )
            raise
        except Exception as e:
            self._logger.error(
                "Domain error during user creation",
                error=str(e),
            )
            raise

class ListUsersUseCase(BaseUseCase):
    """Use case for listing all users."""
    async def execute(self) -> List[UserDTO]:
        async with self._uow:
            try:
                self._logger.info("Starting user listing")
                users = await self._uow.user_repository.find_all()
                self._logger.info("Users retrieved successfully", count=len(users))
                return [user_to_dto(user) for user in users]
            except UserNotFoundException:
                self._logger.error("User not found")
                raise

class UpdateUserUseCase(BaseUseCase):
    """Use case for updating an existing user."""
    def __init__(
        self,
        uow: UnitOfWork,
        logger: ILogger,
        domain_service: UserDomainService,
    ):
        super().__init__(uow, logger)
        self._domain_service = domain_service

    async def execute(self, user_id: int, dto: UserUpdateDTO) -> UserDTO:
        async with self._uow:
            self._logger.set_correlation_id()
            self._logger.info(
                "Starting user update", user_id=user_id, name=dto.name, email=dto.email
            )
```

```
            try:
                existing_user = await self._uow.user_repository.find_by_id(user_id)
                if not existing_user:
                    self._logger.error("User not found for update", user_id=user_id)
                    raise UserNotFoundException(str(user_id))
                # Atualizar a entidade existente diretamente com os novos dados do DTO
                existing_user.update_details(dto.name, dto.email)
                await self._domain_service.validate_user_update(user_id, existing_user)
                saved_user = await self._uow.user_repository.save(existing_user)
                await self._uow.commit()
                saved_user_dto = user_to_dto(saved_user)
                self._logger.info(
                    "User updated successfully",
                    user_id=saved_user_dto.id,
                    name=saved_user_dto.name,
                    email=saved_user_dto.email,
                )
                return saved_user_dto
            except (UserValidationException, UserNotFoundException) as e:
                if isinstance(e, UserValidationException):
                    self._logger.error(
                        "User update validation failed",
                        user_id=user_id,
                        validation_errors=e.validation_errors,
                    )
                else:
                    self._logger.error("User not found for update", user_id=user_id)
                raise
            except Exception as e:
                self._logger.error(
                    "Domain error during user update",
                    user_id=user_id,
                    error=str(e),
                )
                raise

class GetUserUseCase(BaseUseCase):
    """Use case for retrieving a user by ID."""
    async def execute(self, user_id: int) -> UserDTO:
        async with self._uow:
            try:
                self._logger.info("Getting user", user_id=user_id)
                user = await self._uow.user_repository.find_by_id(user_id)
                if not user:
                    self._logger.error("User not found", user_id=user_id)
                    raise UserNotFoundException(str(user_id))
                self._logger.info("User retrieved successfully", user_id=user_id)
                return user_to_dto(user)
            except UserNotFoundException:
                self._logger.error("User not found", user_id=user_id)
                raise

class DeleteUserUseCase(BaseUseCase):
    """Use case for deleting a user by ID."""
    async def execute(self, user_id: int) -> bool:
        async with self._uow:
            try:
                self._logger.info("Starting user deletion", user_id=user_id)
                existing_user = await self._uow.user_repository.find_by_id(user_id)
                if not existing_user:
                    self._logger.error("User not found for deletion", user_id=user_id)
                    raise UserNotFoundException(str(user_id))
                success = await self._uow.user_repository.delete(user_id)
                if success:
                    await self._uow.commit()
                    self._logger.info("User deleted successfully", user_id=user_id)
```

```
    else:
        self._logger.warning("User deletion failed", user_id=user_id)
    return success
except UserNotFoundException:
    self._logger.error("User not found for deletion", user_id=user_id)
    raise
```

33. Arquivo: user_commands.py

```
# ./src/dev_platform/client/cli/user_commands.py
# -*- coding: utf-8 -*-
"""
CLI para gerenciamento de usuários no DEV Platform.
Este módulo define comandos CLI para criar, listar, atualizar, obter e excluir usuários.
"""

import asyncio
import click
import sys
from typing import Optional, List
from dev_platform.infrastructure.config import ConfigurationFacade
from dev_platform.application.user.dtos import UserCreateDTO, UserUpdateDTO, UserDTO
from dev_platform.infrastructure.composition_root import CompositionRoot
from dev_platform.infrastructure.database.unit_of_work import SQLUnitOfWork
from dev_platform.application.ports.logger import ILogger
from dev_platform.infrastructure.logging.structured_logger import StructuredLogger
from dev_platform.domain.exceptions import ConfigurationException


# Logger global para uso em run_async
_LOGGER: ILogger = StructuredLogger()

def run_async(coro) -> None:
    """
    Executa uma corrotina de forma segura em qualquer ambiente.
    Usa run_until_complete em CLI puro e asyncio.run se possível.
    Lança erro amigável se já houver um loop rodando (ex: Jupyter).
    """
    try:
        loop = asyncio.get_event_loop()
        if loop.is_running():
            raise RuntimeError(
                "Já existe um loop de eventos rodando. "
                "Execute este comando em um terminal/CLI puro."
            )
        return loop.run_until_complete(coro)
    except RuntimeError as re:
        _LOGGER.critical(
            f"Runtime error occurred: {re}",
            exception=str(re)
        )
        sys.exit(1)  # Encerra o CLI com erro

class UserCommands:
    def __init__(self, composition_root: CompositionRoot, logger: ILogger):
        self._composition_root = composition_root
        self._logger = logger
        # try:
        #     self._composition_root = CompositionRoot(
        #         environment=config.get("ENVIRONMENT", "production"),
        #         logger=self._logger,
        #         validation_rule_provider=ValidationRuleProvider()
        #     )
        # except ConfigurationException as ce:
        #     self._logger.critical(
        #         f"Configuration error during initialization: {ce}",
        #         exception=str(ce)
        #     )
        #     raise
        # except Exception as e:
        #     self._logger.critical(
        #         f"Unexpected error during initialization: {e}",
```

```
    #         exception=str(e)
    #     )
    #     raise ConfigurationException(
    #         config_key="COMPOSITION_ROOT",
    #         reason=f"Unexpected error: {e}"
    #     )

async def create_user_async(self, name: str, email: str) -> str:
    """
    Cria um novo usuário.
    """
    try:
        async with SQLUnitOfWork() as uow:
            repo = uow.user_repository
            use_case = self._composition_root.create_user_use_case(uow, repo)
            dto: UserCreateDTO = UserCreateDTO(name=name, email=email)
            user: UserDTO = await use_case.execute(dto)
            name_val = getattr(user.name, "value", user.name)
            email_val = getattr(user.email, "value", user.email)
            return f"User created successfully: ID {user.id}, Name: {name_val}, Email: {email_val}"
    except ValueError as e:
        self._logger.warning(f"Validation error: {e}")
        return f"Error: Validation Error: {e}"
    except ConfigurationException as ce:
        self._logger.error(f"Configuration error: {ce}", exception=str(ce))
        return f"Error: Configuration Error: {ce}"
    except Exception as e:
        self._logger.error(f"Error creating user: {e}", exception=str(e))
        return f"Error: {e}"

async def list_users_async(self) -> List[str]:
    """
    Lista todos os usuários.
    """
    try:
        async with SQLUnitOfWork() as uow:
            use_case = self._composition_root.list_users_use_case(uow)
            users: List[UserDTO] = await use_case.execute()
            if not users:
                return ["No users found"]

            result: List[str] = []
            for user in users:
                name_val = getattr(user.name, "value", user.name)
                email_val = getattr(user.email, "value", user.email)
                result.append(
                    f"ID: {user.id}, Name: {name_val}, Email: {email_val}"
                )
            return result
    except ConfigurationException as ce:
        self._logger.error(f"Configuration error: {ce}", exception=str(ce))
        return [f"Error: Configuration Error: {ce}"]
    except Exception as e:
        self._logger.error(f"Error listing users: {e}", exception=str(e))
        return [f"Error: {e}"]

async def update_user_async(
    self, user_id: int, name: Optional[str] = None, email: Optional[str] = None
) -> str:
    """
    Atualiza um usuário existente.
    """
    try:
        async with SQLUnitOfWork() as uow:
            repo = uow.user_repository
            get_use_case = self._composition_root.get_user_use_case(uow, repo)
```

```
            existing_user: UserDTO = await get_use_case.execute(user_id=user_id)
            update_name = name if name is not None else getattr(existing_user.name, "value", existing_use
r.name)
            update_email = email if email is not None else getattr(existing_user.email, "value", existing_user.
email)
            user_update_dto = UserUpdateDTO(name=update_name, email=update_email)
            update_use_case = self._composition_root.update_user_use_case(uow, repo)
            updated_user_entity = await update_use_case.execute(user_id=user_id, dto=user_update_dto)
            name_val = getattr(updated_user_entity.name, "value", updated_user_entity.name)
            email_val = getattr(updated_user_entity.email, "value", updated_user_entity.email)
            return f"User {user_id} updated successfully: ID {updated_user_entity.id}, Name: {name_val}, Ema
il: {email_val}"
        except ConfigurationException as ce:
            self._logger.error(f"Configuration error: {ce}", exception=str(ce))
            return f"Error: Configuration Error: {ce}"
        except Exception as e:
            self._logger.error(f"Error updating user: {e}", exception=str(e))
            return f"Error: {e}"

    async def get_user_async(self, user_id: int) -> str:
        """
        Obtém um usuário pelo ID.
        """
        try:
            async with SQLUnitOfWork() as uow:
                repo = uow.user_repository
                use_case = self._composition_root.get_user_use_case(uow, repo)
                user_entity = await use_case.execute(user_id=user_id)
                if not user_entity:
                    return f"User with ID {user_id} not found."
                name_val = getattr(user_entity.name, "value", user_entity.name)
                email_val = getattr(user_entity.email, "value", user_entity.email)
                return f"User found: ID {user_entity.id}, Name: {name_val}, Email: {email_val}"
        except ConfigurationException as ce:
            self._logger.error(f"Configuration error: {ce}", exception=str(ce))
            return f"Error: Configuration Error: {ce}"
        except Exception as e:
            self._logger.error(f"Error getting user: {e}", exception=str(e))
            return f"Error: {e}"

    async def delete_user_async(self, user_id: int) -> str:
        """
        Exclui um usuário pelo ID.
        """
        try:
            async with SQLUnitOfWork() as uow:
                repo = uow.user_repository
                use_case = self._composition_root.delete_user_use_case(uow, repo)
                success: bool = await use_case.execute(user_id=user_id)
                if success:
                    return f"User {user_id} deleted successfully."
                else:
                    return f"User {user_id} could not be deleted (not found or other issue)."
        except ConfigurationException as ce:
            self._logger.error(f"Configuration error: {ce}", exception=str(ce))
            return f"Error: Configuration Error: {ce}"
        except Exception as e:
            self._logger.error(f"Error deleting user: {e}", exception=str(e))
            return f"Error: {e}"

# Instância única para a sessão da CLI
logger = StructuredLogger()
composition_root = CompositionRoot(environment="production", logger=logger) # Criada uma única ve
z

@click.group()
```

```
def cli():
    pass

@cli.command()
@click.option("--name", prompt="User name")
@click.option("--email", prompt="User email")
def create_user(name: str, email: str):
    """Create a new user."""
    commands: UserCommands = UserCommands(composition_root, logger)

    async def _run_create():
        result: str = await commands.create_user_async(name, email)
        click.echo(result)

    return run_async(_run_create())

@cli.command()
def list_users():
    """List all users."""
    commands: UserCommands = UserCommands(composition_root, logger)

    async def _run_list():
        results: List[str] = await commands.list_users_async()
        for line in results:
            click.echo(line)

    return run_async(_run_list())

@cli.command()
@click.option("--user-id", type=int, prompt="User ID to update")
@click.option(
    "--name",
    prompt="New user name (leave empty to keep current)",
    default="",
    show_default=False,
    help="Novo nome do usuário. Deixe em branco para manter o atual."
)
@click.option(
    "--email",
    prompt="New user email (leave empty to keep current)",
    default="",
    show_default=False,
    help="Novo e-mail do usuário. Deixe em branco para manter o atual."
)
def update_user(user_id: int, name: str, email: str):
    """Update an existing user."""
    commands: UserCommands = UserCommands(composition_root, logger)

    async def _run_update():
        result: str = await commands.update_user_async(
            user_id, name if name else None, email if email else None
        )
        click.echo(result)

    return run_async(_run_update())

@cli.command()
@click.option("--user-id", type=int, prompt="User ID to retrieve")
def get_user(user_id: int):
    """Get a user by ID."""
    commands: UserCommands = UserCommands(composition_root, logger)

    async def _run_get():
        result: str = await commands.get_user_async(user_id)
        click.echo(result)
```

```
    return run_async(_run_get())

@cli.command()
@click.option("--user-id", type=int, prompt="User ID to delete")
def delete_user(user_id: int):
    """Delete a user by ID."""
    commands: UserCommands = UserCommands(composition_root, logger)

    async def _run_delete():
        result: str = await commands.delete_user_async(user_id)
        click.echo(result)

    return run_async(_run_delete())
```

34. Arquivo: user_exceptions.py

```
# ./src/dev_platform/domain/user/user_exceptions.py
# -*- coding: utf-8 -*-
"""
Este módulo define exceções específicas para o domínio de usuários,
permitindo o tratamento de erros de forma consistente e informativa.
"""

from datetime import datetime
from typing import Optional, Dict, Any

class DomainException(Exception):
    """Base exception for all domain-related errors."""

    def __init__(
        self,
        message: str,
        error_code: Optional[str] = None,
        details: Optional[Dict[str, Any]] = None,
    ):
        self.message = message
        self.error_code = error_code or self.__class__.__name__
        self.details = details or {}
        self.timestamp = datetime.now()
        super().__init__(self.message)

    def to_dict(self) -> Dict[str, Any]:
        return {
            "error_code": self.error_code,
            "message": self.message,
            "details": self.details,
            "timestamp": self.timestamp.isoformat(),
        }

    def __str__(self):
        return f"[{self.error_code}] {self.message}"

class UserAlreadyExistsException(DomainException):
    """Raised when trying to create a user with an email that already exists."""

    def __init__(self, email: str):
        self.email = email
        super().__init__(
            message=f"User with email '{email}' already exists",
            error_code="USER_ALREADY_EXISTS",
            details={"email": email},
        )

class UserNotFoundException(DomainException):
    """Raised when a user cannot be found."""

    def __init__(self, identifier: str, identifier_type: str = "id"):
        self.identifier = identifier
        self.identifier_type = identifier_type
        super().__init__(
            message=f"User not found with {identifier_type}: {identifier}",
            error_code="USER_NOT_FOUND",
            details={"identifier": identifier, "identifier_type": identifier_type},
        )

class InvalidUserDataException(DomainException):
    """Raised when user data fails validation."""

    def __init__(self, field: str, value: Any, reason: str):
```

```
        self.field = field
        self.value = value
        self.reason = reason
        super().__init__(
            message=f"Invalid {field}: {reason}",
            error_code="INVALID_USER_DATA",
            details={"field": field, "value": str(value), "reason": reason},
        )

class UserValidationException(DomainException):
    """Raised when user business rules validation fails."""

    def __init__(self, validation_errors: Dict[str, str]):
        self.validation_errors = validation_errors
        errors_summary = ", ".join(
            [f"{field}: {error}" for field, error in validation_errors.items()]
        )
        super().__init__(
            message=f"User validation failed: {errors_summary}",
            error_code="USER_VALIDATION_FAILED",
            details={"validation_errors": validation_errors},
        )

class EmailDomainNotAllowedException(DomainException):
    """Raised when email domain is not in allowed list."""

    def __init__(self, email: str, domain: str, allowed_domains: list):
        self.email = email
        self.domain = domain
        self.allowed_domains = allowed_domains
        super().__init__(
            message=f"Email domain '{domain}' is not allowed. Allowed domains: {', '.join(allowed_domains)}",
            error_code="EMAIL_DOMAIN_NOT_ALLOWED",
            details={
                "email": email,
                "domain": domain,
                "allowed_domains": allowed_domains,
            },
        )

class UserOperationException(DomainException):
    """Raised when a user operation fails."""

    def __init__(self, operation: str, user_id: int, reason: str):
        self.operation = operation
        self.user_id = user_id
        self.reason = reason
        super().__init__(
            message=f"Failed to {operation} user {user_id}: {reason}",
            error_code="USER_OPERATION_FAILED",
            details={"operation": operation, "user_id": user_id, "reason": reason},
        )

# Compatibility aliases (deprecated, use specific exceptions above)
class DomainError(DomainException):
    """Exception for domain-related errors. DEPRECATED: Use DomainException instead."""

    def __init__(self, message: str):
        import warnings

        warnings.warn(
            "DomainError is deprecated. Use DomainException instead.",
            DeprecationWarning,
            stacklevel=2,
        )
        super().__init__(message)
```

```
class ValidationException(DomainException):
    """Exception for validation-related errors. DEPRECATED: Use UserValidationException instead."""

    def __init__(self, message: str):
        import warnings

        warnings.warn(
            "ValidationException is deprecated. Use UserValidationException instead.",
            DeprecationWarning,
            stacklevel=2,
        )
        super().__init__(message)
```

35. Arquivo: validation_rules.py

```
# ./src/dev_platform/domain/validation_rules.py
# -*- coding: utf-8 -*-
"""
Este módulo define regras de validação reutilizáveis para entidades de usuário.
"""

from abc import ABC, abstractmethod
from typing import Optional, List, Set
import re
from datetime import datetime
from dev_platform.domain.user.entities import User

class ValidationRule(ABC):
    """Base class for validation rules."""

    @abstractmethod
    async def validate(self, user: User) -> Optional[str]:
        """
        Validate user according to this rule.
        Returns None if valid, error message if invalid.
        """
        pass

    @property
    @abstractmethod
    def rule_name(self) -> str:
        pass

class EmailFormatAdvancedValidationRule(ValidationRule):
    """Advanced email format validation beyond basic regex."""

    def __init__(self):
        self.pattern = re.compile(
            r"^[a-zA-Z0-9]([a-zA-Z0-9._-]*[a-zA-Z0-9])?@[a-zA-Z0-9]([a-zA-Z0-9.-]*[a-zA-Z0-9])?\.[a-zA-Z]{2,}
$"
        )

    async def validate(self, user: User) -> Optional[str]:
        email = user.email.value

        if not self.pattern.match(email):
            return "Email format is invalid"

        if ".." in email:
            return "Email cannot contain consecutive dots"

        if len(email) > 254:
            return "Email is too long (max 254 characters)"

        local_part, domain_part = email.split("@")

        if len(local_part) > 64:
            return "Email local part is too long (max 64 characters)"

        if len(domain_part) > 253:
            return "Email domain part is too long (max 253 characters)"

        return None

    @property
    def rule_name(self) -> str:
        return "email_format_advanced_validation"
```

```
class NameContentValidationRule(ValidationRule):
    """Validates name content and format."""

    def __init__(self, allowed_chars: Optional[Set[str]] = None):
        if allowed_chars is None:
            allowed_chars = set(
                "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ -'àáâãèéêìíîòóôõùúûçÀÁÂÃÈ
ÉÊÌÍÎÒÓÔÕÙÚÛÇ"
            )
        self.allowed_chars = allowed_chars

    async def validate(self, user: User) -> Optional[str]:
        name = user.name.value

        if name.strip() != name:
            return "Name cannot start or end with whitespace"

        if "  " in name:
            return "Name cannot contain consecutive spaces"

        if any(char.isdigit() for char in name):
            return "Name cannot contain numbers"

        if not all(char in self.allowed_chars for char in name):
            invalid_chars = [char for char in name if char not in self.allowed_chars]
            return f"Name contains invalid characters: {', '.join(set(invalid_chars))}"

        words = name.split()
        if len(words) < 2:
            return "Name must contain at least first and last name"

        for word in words:
            if len(word) < 2:
                return "Each name part must be at least 2 characters long"

        return None

    @property
    def rule_name(self) -> str:
        return "name_content_validation"

class EmailDomainValidationRule(ValidationRule):
    """Validates that email domain is in allowed list."""

    def __init__(self, allowed_domains: List[str]):
        self.allowed_domains = set(domain.lower() for domain in allowed_domains)

    async def validate(self, user: User) -> Optional[str]:
        email_domain = user.email.value.split("@")[1].lower()
        if email_domain not in self.allowed_domains:
            return f"Email domain '{email_domain}' is not allowed. Allowed domains: {', '.join(self.allowed_dom
ains)}"
        return None

    @property
    def rule_name(self) -> str:
        return "email_domain_validation"

class ForbiddenWordsValidationRule(ValidationRule):
    """
    Valida se o nome do usuário contém palavras específicas proibidas.
    """

    def __init__(self, forbidden_words: List[str]):
        self.forbidden_words = [word.lower() for word in forbidden_words]
```

```
    async def validate(self, user: User) -> Optional[str]:
        for word in self.forbidden_words:
            if word in user.name.value.lower():
                return f"Name contains forbidden word: {word}"
        return None

    @property
    def rule_name(self) -> str:
        return "forbidden_words_validation"

class NameProfanityValidationRule(ValidationRule):
    """
    Valida se o nome do usuário contém palavrões ou termos ofensivos.
    """

    def __init__(self, forbidden_words: List[str]):
        self.forbidden_words = [word.lower() for word in forbidden_words]

    async def validate(self, user: User) -> Optional[str]:
        name_lower = user.name.value.lower()
        for word in self.forbidden_words:
            if word in name_lower:
                return f"Name contains forbidden word: {word}"
        return None

    @property
    def rule_name(self) -> str:
        return "name_profanity_validation"

class BusinessHoursValidationRule(ValidationRule):
    """Validates if operation is within business hours."""

    def __init__(self, business_hours_only: bool = False):
        self.business_hours_only = business_hours_only

    async def validate(self, user: User) -> Optional[str]:
        if not self.business_hours_only:
            return None

        now = datetime.now()
        if now.weekday() >= 5:
            return "User registration only allowed during business days"

        if now.hour < 9 or now.hour >= 17:
            return "User registration only allowed during business hours (9 AM - 5 PM)"

        return None

    @property
    def rule_name(self) -> str:
        return "business_hours_validation"
```

36. Arquivo: value_objects.py

```
# ./src/dev_platform/domain/user/value_objects.py
# -*- coding: utf-8 -*-
"""
Este módulo define os Value Objects para o domínio de usuários, incluindo Email e UserName.
"""
from dataclasses import dataclass
import re


@dataclass(frozen=True)
class Email:
    value: str

    def __post_init__(self):
        if not self._is_valid():
            raise ValueError(f"Invalid email format: {self.value}")

    def _is_valid(self) -> bool:
        pattern = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        return bool(re.match(pattern, self.value))

@dataclass(frozen=True)
class UserName:
    value: str

    def __post_init__(self):
        value = self.value.strip()
        if not value or len(value) < 3:
            raise ValueError("Name must be at least 3 characters long")
        if len(self.value) > 100:
            raise ValueError("Name cannot exceed 100 characters")
```